TECHNICAL REPORT · RTL DESIGN & FUNCTIONAL VERIFICATION

SPI Slave CDC Bridge

RTL 설계 및 기능 검증 최종 기술 보고서

Verilog-2001 · SPI Mode 0 · 8-bit MSB-first · Async FIFO + 2FF / Pulse Synchronizer
ACLK 100 MHz · SCLK 25 MHz (ACLK ÷ 4) · Vivado 2025.1 XSim

작성 목적	Logic Synthesis / Physical Implementation / PD 직무 지원용 기술 포트폴리오 — 실제 RTL 개발·디버깅·검증 과정을 실패와 수정까지 포함해 기록
대상 RTL	`spi_bridge_top.v` (1,293 lines, 8 modules, SHA-256 f7cfaff9…)
대상 TB	`tb_spi_bridge_top.v` (1,090 lines, SHA-256 96d3c95b…)
최종 시뮬레이션 증거	LOG-13 (Vivado 2025.1 XSim, 2026-10-21 13:12 로그) — 마지막 run: **ALL TESTS PASSED @ 44,896 ns**, check 68 PASS / 0 FAIL 확인
증거 수집일	2026-10-21 (증거 묶음 SPI_RTL_Verification_Collected_Evidence_20261021.zip)
이 보고서의 범위	RTL 코드 분석 + 제공된 로그·스크린샷·합성 리포트의 정리. **이 보고서 작성 과정에서 새 시뮬레이션·합성·lint·CDC·STA는 수행하지 않았습니다** (작업 환경에 시뮬레이터 없음).

산출물 구성

A	최종 기술 보고서 (이 문서) — 1~10장
B	Debugging Issue Tracker — 부록 B
C	Verification Matrix — 8.1절
D	Portfolio Summary (4쪽) — 별도 PDF
E	Evidence Index — 부록 E (전체 SHA-256 은 증거 묶음의 README_Evidence_Index.txt)

0 문서 정보와 증거 등급

0.1 이 보고서를 읽는 방법

모든 주장에는 근거 ID(예: `LOG-13:L2929`, `RTL-F:L874`, `SS-27`)와 증거 등급을 붙였습니다. 등급은 “무엇이 실제로 확인되었는가”를 구분하기 위한 것이며, 확인되지 않은 것은 확인된 것처럼 쓰지 않았습니다.

등급	의미	예
확인	제공된 로그·RTL·TB·리포트에서 직접 읽은 사실	LOG-13 의 `ALL TESTS PASSED`, RTL 의 `BUFG` 인스턴스
추정	증거로부터 논리적으로 도출했으나 직접 확인 문서가 없는 해석	디버깅 원인 추론, 특정 RUN 이 사용한 정확한 RTL 리비전
재구성	로그 값·RTL 수식으로 작성자가 다시 그린 파형/표 (스크린샷이 아님)	그림 7-2, 7-3
사용자 보고	증거 파일에는 없고 프로젝트 소유자가 대화에서 말한 사실	합성 SDC 의 1 GHz 가 교수님 규칙이라는 점
미확인	증거에 값이 보이지 않아 알 수 없는 항목	판독 불가 신호값 → “확인 불가”로 표기
미수행	수행한 증거가 없는 작업	lint, CDC/RDC, STA(최종 RTL), LEC, P&R, 정량 coverage
예정	앞으로 할 일 (완료 아님)	10장 next step
예시(가상)	설명용 가상 값 — 이 보고서에서는 사용하지 않았음	—

0.2 보고서가 하지 않는 주장

- PASS 개수(68)를 coverage 퍼센트로 환산하지 않습니다. **정량 Coverage 는 측정하지 않았습니다.**
- CDC 가 모두 해결되었다고 주장하지 않습니다. 최종 RTL 에는 아직 열린 CDC 항목이 2개 있습니다(3.5절).
- 합성(Genus) 결과는 **최종 RTL 과 다른 리비전**의 netlist 에서 나온 것이며, 최종 RTL 의 합성·STA 결과가 아닙니다(3.6절).
- Vivado 파형 스크린샷 중 확대 없이는 읽을 수 없는 것은 쓰지 않았고, 대신 로그 값과 RTL 로 다시 그린 파형을 “재구성”으로 표기해 넣었습니다.

0.3 목차

1 개요 (Overview) 5
2 RTL 아키텍처 — 모듈별 상세 7
3 Clock / Reset / CDC 분석 11
4 Debugging History (RTL-001 ~ RTL-008, TB-001 ~ TB-005) 14
5 검증 전략과 Testbench 구조 23
6 TEST 1 ~ TEST 10 상세 25
7 시뮬레이션 결과와 증거 30
8 Coverage, 한계, Verification Matrix 32
9 Lessons Learned 33
10 결론과 다음 단계 34
A 부록 A — RTL 발췌 35
B 부록 B — Debugging Issue Tracker 36
C 부록 C — 최종 실행 PASS 로그 전문(요약) 37
E 부록 E — Evidence Index 38

0.4 Source Inventory 와 Debugging Timeline

보고서를 쓰기 전에 증거 묶음 전체를 목록화하고 시간순으로 정렬했습니다. 증거 묶음 자체가 “실행에 쓰인 정확한 파일”을 보장하지 않는다고 명시하므로(`README_Evidence_Index.txt`), 아래 표의 “사용” 열은 확인된 범위만 적었습니다.

ID	구분	파일(요약)	내용	이 보고서에서의 사용
RTL-F	최종 RTL	`spi_bridge_top.v`	8 모듈 단일 파일, 1,293 lines. 전부 읽음 확인	2·3장 설명의 기준
RTL-O / RTL-1	이전 RTL	`spi_bridge_all_in_one.v`, `…_fixed.v`, 분리 모듈 14개(Spi_*.v 등), TX-hold 리비전(로그 124556 에 포함, CRLF)	diff 로 변경점 확인 확인	4장 RTL-001~007 의 before/after
TB-F	최종 TB	`tb_spi_bridge_top.v`	1,090 lines, TEST 1–10. 전부 읽음 확인	5·6장
TB-V	TB 이력	test1_to_test6/7/7_rxvalid_check_fixed/8, tb_async_fifo, tb_register_block 등	진화 과정	4장 TB-00x
LOG-13	최종 로그	`붙여넣은 텍스트(1)(20261021-131223).txt`	3,287 lines, XSim 13 회 run 연속 기록. 마지막 run = L2929–3287	7장의 주 증거
LOG-xx	중간 로그	093704/093905/094912(queue TB), 095412, 095946/100015/121810(spi_tx_burst), (4)(5)(7)(8) 누적 Vivado 로그	실패·수정 증거	4장
SS-xx	스크린샷	PNG 31장: SimVision(10/14–15), Vivado XSim(10/21)	파형 판독 가능한 것만 파형 영역으로 잘라 사용(그림 4-2 ~ 4-11)	4장
SPEC	명세	8-bit_SPI_Master_Slave_Async_FIFO_Specification(1).pdf (3 pp)	SCLK 독립 25 MHz, FIFO 8×16, XC7Z010 등	1·3장(명세 대비 차이)
REV	리뷰 문서	SPI_Bridge_Review_Report.pdf, SPI_Bridge_TX_Update_Report.pdf, 00_Master_TOC_v0.1.pdf	AI 어시스턴트가 작성한 리뷰. **도구 증거 아님** (Python 2-state 해석기 사용분 포함)	참고만, 근거로 쓰지 않음
SYN	합성	Genus 22.17 리포트 41개 + SDC 2개	2026-10-20 06:09 합성, 다른 RTL 리비전	3.6장, 8장(참고)

그림 4-1 개발·디버깅 타임라인 (증거 파일명의 타임스탬프 기준, 시간대 미확인)



LOG-13 (붙여넣은 텍스트(1)(20261021-131223).txt) 안의 RUN 1–13



RUN 4→5: TX hold register 도입 (RTL-006). RUN 9→10: SCLK 도메인 async reset (RTL-007). RUN 11→12: TEST7 RX_VALID 체크를 55 ns 펄스로 한정 (TB-004). RUN 12→13: TEST 8–10 추가.
[확인] RUN 경계/결과는 LOG-13 의 "Running xsim" 구분과 PASS/FAIL 줄. [추정] 각 RUN 에 사용된 정확한 RTL 파일 리비전(증거 묶음은 "정확히 그 파일"을 보장하지 않음, README_Evidence_Index.txt).
[확인] RUN 8 은 TEST1–5 를 디버그 출력 포함 TB(라벨 TEST 1: RX Sequential)로 실행한 별도 버전이며 최종 TB 와 동일하지 않음.

0.5 시간순 요약 (증거 파일 타임스탬프 기준)

일시	도구	사건	증거
10/14 03:55–04:24	Cadence Xcelium / SimVision	tb_register_block FAIL, tb_async_fifo TIMEOUT(`xmsim *E, RNFNSH`), FIFO 파형에서 `o_empty` 고정·`o_rd_data`=xx	SS-00, SS-02..05 확인
10/15	SimVision	FIFO level 파형 관찰	SS-07
10/16–10/20	Genus 22.17	합성 도달, 1 GHz SDC, slack +66.6 ps (ACLK)	SYN-* 확인 / 날짜 일부 사용자 보고
10/21 01:03–02:19	Vivado 2025.1 XSim	첫 top 레벨 run: STATUS 읽기값 X, `FAIL: TIMEOUT -- only 0/6 RX frames checked` (2 ms)	SS-08..23, LOG-(4)(5)(7)(8)
10/21 09:37–09:49	XSim	queue-scoreboard TB: `LOSS of 1 byte(s): expected 01 got 02`	LOG-093905
10/21 09:53–12:21	XSim	spi_tx_burst: MISO 0x33×16 → TX hold 도입 후 0x11/0x22/0x33 PASS	LOG-100015, 121810
10/21 12:18–12:29	XSim	TX hold 신호 파형 확인(hold_valid, state, bit_cnt, w_tx_consume)	SS-25..28
10/21 12:45–13:12	XSim	최종 TB RUN 1–13: FAIL 5 회(RUN1–4 묶음, 9, 11) → 수정 → ALL TESTS PASSED	LOG-13

1 개요 (Overview)

1.1 프로젝트 목적과 범위

SPI Slave CDC Bridge 는 SPI 로 들어오는 8-bit 데이터를 100 MHz 버스 도메인(ACLK)의 레지스터 인터페이스로 전달하고, 반대로 버스에서 쓴 데이터를 SPI MISO 로 내보내는 브리지입니다. 서로 다른 클럭(ACLK 100 MHz / SCLK 25 MHz)의 경계를 **비동기 FIFO(Gray pointer + 2FF)**, **pulse synchronizer**, **level 2FF synchronizer** 로 넘기는 것이 설계의 핵심이며, 이 보고서는 그 RTL 이 어떻게 만들어졌고 어떤 오류를 거쳐 어떤 검증을 통과했는지를 기록합니다.

프로토콜	SPI Mode 0 (CPOL=0, CPHA=0), 8-bit, MSB-first. MOSI 는 SCLK 상승 edge 에서 slave 가 샘플, MISO 는 하강 edge 에서 갱신 확인 (RTL-F spi_slave)
클럭	ACLK 100 MHz(TB 10 ns 주기), SCLK = ACLK ÷ 4 = 25 MHz. `sclk_gen` 이 2 ACLK 마다 toggle 후 `BUFG` 를 거침 확인
리셋	외부 active-low `ARESETn`, 도메인별 `reset_sync` 2개 (assert 비동기 / release 2FF 동기) 확인
FIFO	RX 16×8 (SCLK→ACLK), TX 4×8 (ACLK→SCLK) — 명세의 8×16 과 다름(1.4절)
레지스터	0x00 RX_DATA · 0x04 STATUS · 0x08 CONTROL · 0x0C FIFO_LEVEL · 0x10 TX_DATA
개발 환경	Vivado 2025.1 / XSim (최종 검증). 이전: Cadence Xcelium/SimVision (10/14–15). 합성: Cadence Genus 22.17 (참고, 3.6절)

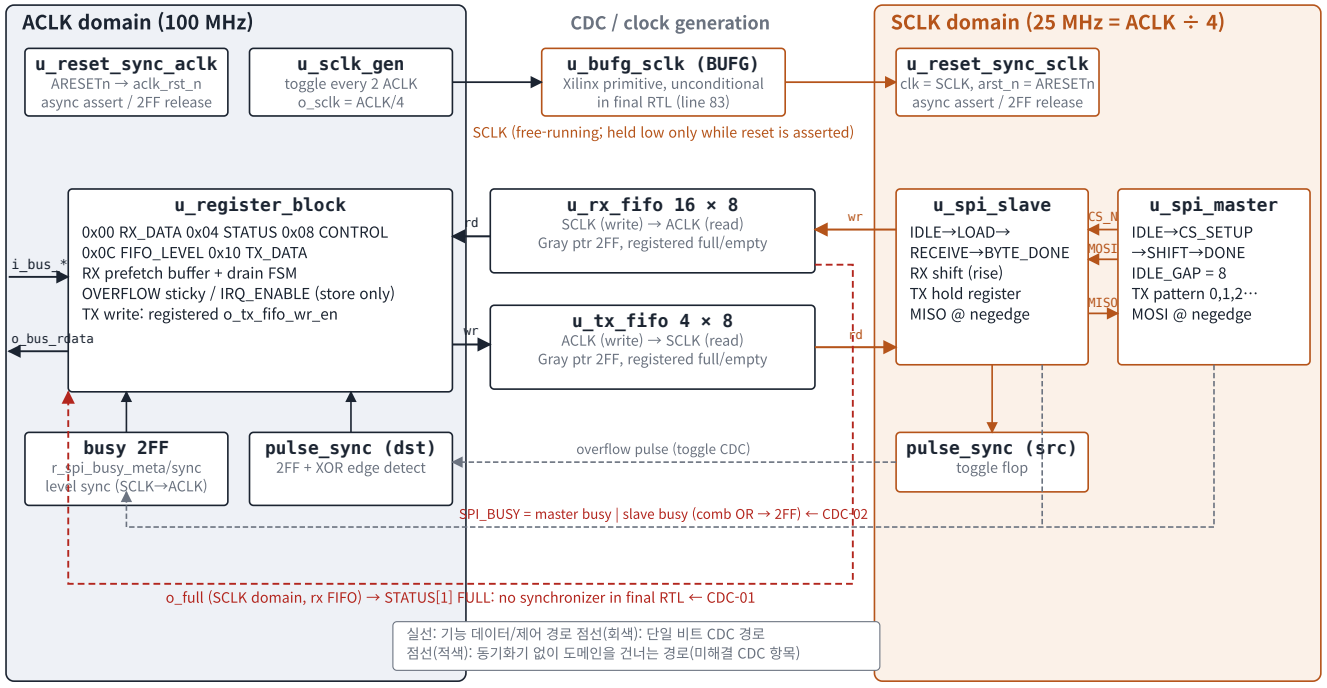


그림 1-1 spi_bridge_top 블록도 (최종 RTL spi_bridge_top.v 기준, 인스턴스명은 RTL과 동일)

1.2 모듈 구성

모듈	RTL-F 라인	도메인	역할
`spi_bridge_top`	7–249	ACLK / SCLK	최상위. 클럭·리셋 생성, 모듈 연결, CDC 경계(busy 2FF, overflow pulse_sync)
`spi_master`	255–451	SCLK	내장 TX/RX 패턴 생성용 SPI master (IDLE/CS_SETUP/SHIFT/DONE). 테스트 구동원
`spi_slave`	485–789	SCLK	브리지 본체의 SPI 쪽. RX shift → RX FIFO, TX FIFO → hold register → MISO
`async_fifo`	795–950	wr_clk / rd_clk	Gray pointer 비동기 FIFO. RX(16)/TX(4) 로 2회 인스턴스
`register_block`	956–1143	ACLK	버스 레지스터, RX prefetch + drain FSM, 상태/레벨/overflow

모듈	RTL-F 라인	도메인	역할
`pulse_sync`	1149–1208	src→dst	toggle + 2FF + XOR 펄스 동기 (overflow)
`reset_sync`	1214–1250	도메인별	async assert / 2FF sync release
`sclk_gen`	1256–1293	ACLK	SCLK 분주기 (÷4)

1.3 이 보고서의 핵심 결과 한 장 요약

항목	결과	등급
최종 TB (TEST 1–10) 시뮬레이션	XSim 에서 `ALL TESTS PASSED` @44,896 ns, check 68 PASS / 0 FAIL	확인
디버깅된 RTL 결함	FIFO 조합 루프, 연산자 오타(- ↔ ~), 포인터 동기 상수 0, master 세미콜론, TX 다중 pop, SCLK 도메인 sync reset (총 7건 수정)	확인 (RTL-001..007)
남아 있는 RTL 결함	RX prefetch double-pop (RTL-008) — 최종 TB 로는 검출 불가	확인 (로그+코드)
열린 CDC 항목	`rx_full` 직접 연결(CDC-01), busy 조합 OR 후 2FF(CDC-02)	확인 (코드)
Lint / CDC·RDC / STA / LEC / P&R (최종 RTL)	증거 없음	미수행
정량 Coverage	측정하지 않음	미수행

1.4 명세와 최종 RTL 의 차이

최종 RTL 이 명세(SPEC, 3 pp)와 다른 부분을 먼저 밝힙니다. 보고서의 모든 분석은 **명세가 아니라 실제 RTL** 을 기준으로 합니다.

항목	명세(SPEC)	최종 RTL (RTL-F)	영향
SCLK	독립된 25 MHz 클럭	`sclk_gen` 이 ACLK ÷ 4 로 **생성**하는 generated clock (`o_sclk`→BUFG). CS_N=1 구간에도 toggle(free-running), 리셋 중에만 low 고정	CDC 해석이 달라짐: 두 도메인은 비동기가 아니라 **동일 소스의 주기 관계**. 단 리셋 중 SCLK 정지 → RDC 위험(RTL-007)
FIFO 크기	8-bit × 16	RX 16×8, **TX 4×8** (`ADDR_WIDTH` 파라미터)	TEST 5 는 TX FIFO 4개 수락 후 full 을 검증
IRQ	—(명세에서 확인 불가)	CONTROL[2] `IRQ_ENABLE` 는 저장/readback 만. top 에 IRQ 포트 없음	TEST 10 은 readback 만 검증
타깃	XC7Z010	BUFG 무조건 인스턴스(line 83). 합성 SDC 는 `XILINX_BUFG` 미정의 가정	3.6절

2 RTL 아키텍처 — 모듈별 상세

각 모듈을 같은 10개 항목으로 정리했습니다. 라인 번호는 최종 RTL(`spi_bridge_top.v`) 기준이며, 설명은 모두 코드를 직접 읽은 내용입니다 확인. "검증 증거"는 최종 로그 LOG-13 에서 해당 기능을 간접·직접 검사하는 TEST 를 가리킵니다.

2.1 spi_bridge_top · RTL-F 7–249

1. 목적	클럭·리셋 생성, 8 모듈 연결, CDC 경계(busy 2FF, overflow pulse_sync) 구성. 외부에는 버스형 레지스터 포트와 debug SPI 신호만 노출.
2. 인터페이스	in: `ACLK`, `ARESETn`, `i_bus_addr[7:0]`, `i_bus_wr_en`, `i_bus_wdata[7:0]`, `i_bus_rd_en`. out: `o_bus_rdata[7:0]`, `o_overflow`, `o_rx_valid`, `o_dbg_sclk/cs_n/mosi/miso`. 포트 주석: "placeholder until an AXI4-Lite wrapper is added".
3. 클럭 / 리셋	ACLK 도메인: `aclk_rst_n` ← `u_reset_sync_aclk`. SCLK 도메인: `sclk_rst_n` ← `u_reset_sync_sclk` (clk=SCLK, arst_n=ARESETn). SCLK 는 `sclk_gen` → `BUFG` (line 83).
4. 내부 구조	`u_spi_master` →(MOSI, CS_N)→ `u_spi_slave` →(MISO)→master. slave 가 RX FIFO 쓰기, TX FIFO 읽기. 버스 쪽은 `u_register_block`. FIFO: `u_rx_fifo` (ADDR_WIDTH=4), `u_tx_fifo` (ADDR_WIDTH=2).
5. 상태·주요 레지스터	`r_spi_busy_meta/sync` (2FF, lines 206–217), `w_spi_busy = w_spi_busy_master || w_spi_busy_slave`.
6. 타이밍 동작	master/slave 가 같은 칩 안에서 연결되므로 SPI 신호는 내부 배선이며 debug 포트로만 관찰됨. 외부 SPI 마스터 입력 포트는 **없음**.
7. CDC 관련	SCLK→ACLK: overflow pulse(`pulse_sync`), busy(2FF), **rx_full(동기기 없음, CDC-01)**. ACLK→SCLK: TX FIFO(Gray). 상세 3.4절.
8. 검증 증거	전 TEST 가 top 포트(`i_bus_*`, `o_dbg_*`)로 구동·관찰 (TB-F). TEST 5 는 `dut.w_tx_fifo_wr_en` 계층 참조 사용.
9. 알려진 문제·한계	① busy 가 조합 OR 이후 2FF (glitch 가능성, CDC-02). ② `rx_full` 직접 연결(CDC-01). ③ BUFG 무조건 인스턴스 — 비 Xilinx 흐름 이식 시 수정 필요. ④ master 의 `o_rx_captured*` 는 미연결.
10. 합성 관점	Genus 합성은 다른 RTL 리비전(3.6절). 이 top 은 `XILINX_BUFG` 가드가 없으므로 합성 SDC 가정과 불일치 확인.

2.2 spi_master · RTL-F 255–451

1. 목적	SCLK 도메인에서 CS_N, MOSI 를 구동해 slave 에 프레임을 보내는 내장 SPI master. TX 패턴은 0부터 자동 증가하는 바이트 — RX 경로 테스트용 자극원.
2. 인터페이스	in: `SCLK`, `rst_n`, `i_miso`. out: `o_cs_n`, `o_mosi`, `o_busy`, `o_rx_captured[7:0]`, `o_rx_captured_valid` (마지막 둘은 top 에서 미연결). 파라미터 `IDLE_GAP`=8'd8.
3. 클럭 / 리셋	`posedge SCLK or negedge rst_n` (async reset). MOSI 는 negedge 에서 launch.
4. 내부 구조	4-state FSM + shift register + 프레임 간 gap 카운터.
5. 상태·주요 레지스터	`S_IDLE / S_CS_SETUP / S_SHIFT / S_DONE` (2-bit). IDLE 에서 IDLE_GAP SCLK 동안 CS_N=1 유지 후 프레임 시작.
6. 타이밍 동작	프레임 = **19 SCLK cycle = 760 ns** (TEST 3 의 연속 TX 프레임 완료 시각 6,246 → 7,006 → 7,766 ns, 간격 760 ns 와 일치) 확인.
7. CDC 관련	동일 SCLK 도메인 내부 — CDC 없음.
8. 검증 증거	TEST 1 (프레임 6개 RX), TEST 4 (22 프레임 backlog), TEST 6/7 (프레임 중간 reset).
9. 알려진 문제·한계	RTL-004/005 에서 수정된 두 결함(세미콜론, captured_valid) 존재했음(4장). 출력 일부가 미사용이라 top 레벨 TB 로는 검증되지 않음 확인. 외부 master 입력 없음.
10. 합성 관점	합성에서는 미연결 출력이 제거될 수 있음 (Genus 리포트에서 별도 확인하지 않음 미확인).

2.3 spi_slave · RTL-F 485–789

1. 목적	SPI Mode 0 slave. MOSI 를 8 bit 모아 RX FIFO 에 쓰고, TX FIFO 의 바이트를 hold register 를 거쳐 MISO 로 내보낸다.
2. 인터페이스	in: `SCLK`, `rst_n`, `i_cs_n`, `i_mosi`, `i_rx_fifo_full`, `i_tx_fifo_rd_data[7:0]`, `i_tx_fifo_empty`. out: `o_miso`, `o_rx_wr_en`, `o_rx_wr_data[7:0]`, `o_rx_overflow_pulse`, `o_tx_fifo_rd_en`, `o_spi_busy`.
3. 클럭 / 리셋	posedge/negedge SCLK 모두 `or negedge rst_n` (async reset; RTL-007 수정). negedge 블록은 별도 클럭 도메인이 아니라 같은 SCLK 의 반대 edge.
4. 내부 구조	FSM + RX shift(`r_shift_in`) + TX shift(`r_shift_tx`) + TX hold 경로(`r_tx_hold`, `r_tx_hold_valid`, `r_tx_pop_d`, `r_tx_cur_from_hold`).
5. 상태·주요 레지스터	`S_IDLE → S_LOAD → S_RECEIVE → S_BYTE_DONE` (→ S_RECEIVE 또는 S_IDLE). `o_spi_busy = (state != S_IDLE)`. `w_tx_pop = !empty && !hold_valid && !pop_d`, `w_tx_consume = RECEIVE && bit_cnt==0 && !cs_n && cur_from_hold`.
6. 타이밍 동작	MOSI: posedge SCLK 샘플. MISO: negedge SCLK 갱신. FIFO 읽기 데이터는 1 cycle 뒤 hold 에 저장(`r_tx_pop_d` 로 정렬). `o_rx_wr_en` / `o_rx_overflow_pulse` 는 `state==S_BYTE_DONE` 에서 조합 생성 — overflow 는 FIFO full 일 때.
7. CDC 관련	RX FIFO write 측, TX FIFO read 측 모두 SCLK 도메인 → 직접 연결이 정상. `i_rx_fifo_full` 은 FIFO wr 도메인(SCLK) 신호라 안전.
8. 검증 증거	TEST 1–3 (RX/TX end-to-end), TEST 4 (overflow), TEST 6/7 (reset 복구), TX_DEBUG 로그(LOG-13) 255 샘플.
9. 알려진 문제·한계	RTL-006(TX 다중 pop) 수정 이력. hold 도입 후 **TX FIFO 가 비어 있다가 늦게 채워질 때**의 프레임 경계 타이밍은 TEST 3 의 3바이트 burst 범위에서만 검증.
10. 합성 관점	Genus netlist 에는 이 slave 의 구현이 포함되었으나 리비전이 다름(3.6절).

2.4 async_fifo · RTL-F 795–950

1. 목적	Gray pointer 기반 비동기 FIFO. RX(16×8), TX(4×8) 로 재사용.
2. 인터페이스	`wr_clk, wr_rst_n, i_wr_en, i_wr_data, o_full` / `rd_clk, rd_rst_n, i_rd_en, o_rd_data, o_empty, o_rd_level[4:0]`. 파라미터 `DATA_WIDTH`=8, `ADDR_WIDTH` (RX 4 / TX 2), `DEPTH = 1<<ADDR_WIDTH` (line 841).
3. 클럭 / 리셋	wr 쪽 `wr_clk`/`wr_rst_n`, rd 쪽 `rd_clk`/`rd_rst_n`. 포인터 레지스터는 각 도메인 reset.
4. 내부 구조	이진·Gray 포인터(각 ADDR_WIDTH+1 bit), 메모리 배열, 반대 도메인 Gray 포인터의 2FF 동기기, Gray→binary 변환(`o_rd_level`).
5. 상태·주요 레지스터	`r_full`, `r_empty` (registered flag), `o_rd_data` registered, `o_rd_level = gray2bin(wr_gray_rclk) - rd_bin`.
6. 타이밍 동작	`full`: `w_wr_ptr_gray_next == {~rd_gray[4:3], rd_gray[2:0]}` (line 874, ADDR_WIDTH=4 일 때). `empty`: `w_rd_ptr_gray_next == r_wr_ptr_gray_rclk` (line 912). 읽기 데이터는 한 cycle 뒤 유효(register 출력).
7. CDC 관련	모든 포인터 교차가 Gray + 2FF. 데이터 배열은 포인터 handshake 로 보호(데이터 버스 개별 동기화 없음). 3.4절.
8. 검증 증거	TEST 8 (full/empty, wrap, Gray 단일 비트 전이 checker, 32 write/32 read), TEST 9 (128 byte 동시 R/W), 그리고 TEST 1/3/4 의 top 경로. **TEST 8/9 는 DUT 내부가 아니라 TB 가 따로 만든 FIFO 인스턴스**를 사용 확인.
9. 알려진 문제·한계	RTL-001/002/003 에서 3 결함 수정. full/empty 식은 `ADDR_WIDTH` 로 파라미터화되어 있어(`{~rd_gray[AW:AW-1], rd_gray[AW-2:0]}`) RX(4)/TX(2) 에 모두 적용됨 확인. 메모리 쓰기는 reset 이 없는 별도 always 블록(데이터 배열은 reset 대상 아님). 동시 read/write·pointer wrap 은 TEST 8/9 에서 검사.
10. 합성 관점	메모리 배열은 RTL 상 register array — Genus 면적에서 u_rx_fifo 2,179.5 / u_tx_fifo 720.8 로 최대 비중 확인(다른 리비전). FPGA 라면 BRAM/LUTRAM 추론 여부 미확인.

2.5 register_block · RTL-F 956–1143

1. 목적	버스 레지스터 인터페이스: RX 데이터 prefetch, STATUS/CONTROL/FIFO_LEVEL, TX 쓰기, overflow sticky.
2. 인터페이스	in: `ACLK, rst_n, i_bus_addr/wr_en/wdata/rd_en`, RX/TX FIFO 상태·데이터, `i_spi_busy_sync`, `i_rx_overflow_pulse_sync`. out: `o_bus_rdata`, `o_rx_fifo_rd_en`, `o_tx_fifo_wr_en/wr_data`, `o_overflow`, `o_rx_valid`.
3. 클럭 / 리셋	ACLK, **synchronous reset** (`if (!rst_n)`) — ACLK 이 항상 동작하므로 허용. SCLK 도메인과 달리 문제 없음.
4. 내부 구조	RX prefetch buffer(`r_rx_data_buf`, `rx_data_buf_valid`) + drain FSM(`r_drain_active`) + TX 쓰기 레지스터 + 조합 read mux.
5. 상태·주요 레지스터	맵: 00 RX_DATA, 04 STATUS, 08 CONTROL, 0C FIFO_LEVEL, 10 TX_DATA. STATUS: [0] EMPTY, [1] FULL, [2] SPI_BUSY, [3] RX_VALID, [4] OVERFLOW. CONTROL: [0] FIFO_CLEAR, [1] OVF_CLEAR, [2] IRQ_ENABLE.
6. 타이밍 동작	RX: `w_rx_prefetch = !valid && !empty && !drain` → `o_rx_fifo_rd_en`; 다음 cycle(`r_rx_fifo_rd_en_d`)에 FIFO 출력 래치, valid=1. RX_DATA 읽으면 valid=0. TX: `o_tx_fifo_wr_en <= wr_en && addr==0x10 && !i_tx_fifo_full` (**registered**). overflow: 동기된 펄스가 set 우선, OVF_CLEAR 로 clear.
7. CDC 관련	입력 `i_rx_fifo_full` 은 SCLK 도메인 신호가 그대로 STATUS[1] 로 들어옴(CDC-01). `i_spi_busy_sync`, `i_rx_overflow_pulse_sync` 는 동기 후 신호.
8. 검증 증거	TEST 1/4/10 (레지스터), TEST 5 (TX 쓰기), TEST 3 (burst).
9. 알려진 문제·한계	**RTL-008**: `w_rx_prefetch` 에 `!r_rx_fifo_rd_en_d` 가 없어 FIFO 에 2개 이상 있을 때 연속 2 pop → 1 byte 손실 (LOG-093905 에서 재현). 최종 RTL 에 남아 있음. `FIFO_CLEAR` 는 **RX FIFO 만** drain 하며 TX FIFO/hold 는 건드리지 않음. IRQ_ENABLE 은 출력 없음.
10. 합성 관점	읽기 mux 가 조합이라 `o_bus_rdata` 출력 경로가 길다 — 합성 critical path 는 이 모듈이 아니라 `i_bus_addr[0]` → `w_tx_fifo_wr_en` (다른 리비전, 조합 write enable)였음 확인.

2.6 pulse_sync · RTL-F 1149–1208

1. 목적	한 도메인의 1-cycle 펄스를 다른 도메인의 1-cycle 펄스로 변환 (overflow 이벤트 전달).
2. 인터페이스	in: `src_clk, src_rst_n, i_src_pulse, dst_clk, dst_rst_n`. out: `o_dst_pulse`.
3. 클럭 / 리셋	양 도메인 모두 async reset (`or negedge *_rst_n`) — RTL-007 후속 변경.
4. 내부 구조	src: toggle flop `r_src_toggle`. dst: `r_dst_meta → r_dst_sync → r_dst_sync_d` + XOR.
5. 상태·주요 레지스터	`o_dst_pulse = r_dst_sync ^ r_dst_sync_d`.
6. 타이밍 동작	src 펄스 → toggle 반전 → dst 에서 2FF 후 1 dst_clk 폭 펄스. 지연 약 2–3 dst cycle.
7. CDC 관련	이 모듈이 CDC 구조 자체. 전제: src 펄스 간격이 동기 지연보다 충분히 길어야 함(연속 펄스 소실 가능 — 설계 가정 추정).
8. 검증 증거	TEST 4 의 `OVERFLOW sticky=1`, `o_overflow` 출력, TEST 6/7 의 overflow 리셋 체크.
9. 알려진 문제·한계	고속 연속 overflow 펄스에서의 합병 동작은 검증하지 않음 미확인.
10. 합성 관점	toggle flop → 첫 동기 flop 경로가 CDC 제약 대상. Genus 의 `set_max_delay -datapath_only` 는 비활성(TUI-64) 확인.

2.7 reset_sync · RTL-F 1214–1250

1. 목적	도메인별 reset: assert 는 즉시(비동기), release 는 해당 클럭에 동기.
2. 인터페이스	in: `clk, arst_n`. out: `rst_n_sync`.
3. 클럭 / 리셋	ACLK 용과 SCLK 용 2 인스턴스. 둘 다 입력은 `ARESETn`.
4. 내부 구조	2-FF (`r_meta_ff`, `rst_n_sync`), D 입력은 1'b1.
5. 상태·주요 레지스터	리셋 시 둘 다 0, 해제 후 2 clk 지나 `rst_n_sync`=1.
6. 타이밍 동작	해제 지연 = 2 clk edge. SCLK 용은 **SCLK 가 다시 toggle 하기 시작해야** 해제됨(sclk_gen 은 aclk_rst_n 이 풀려야 시작).

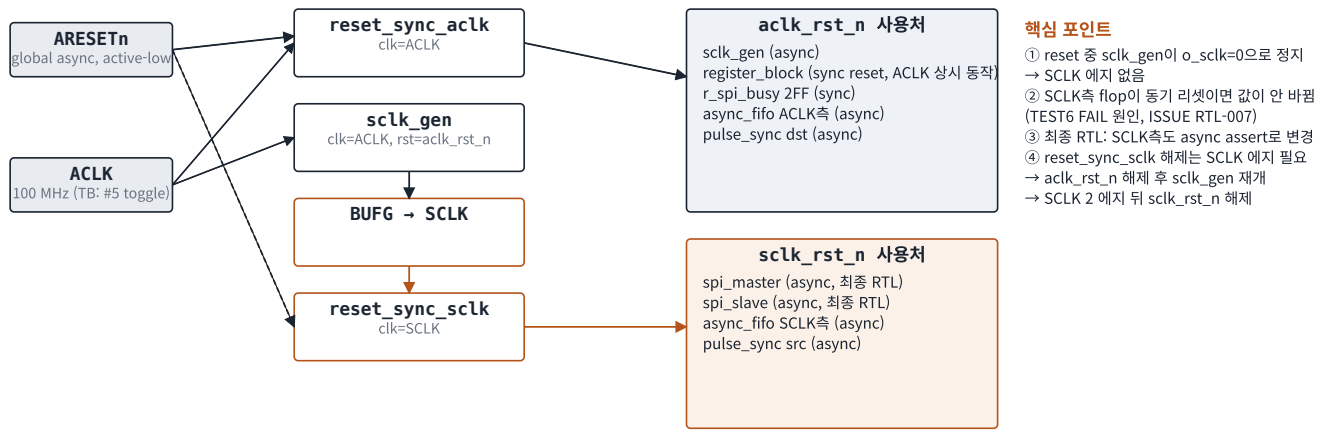
7. CDC 관련	비동기 reset 입력(ARESETn) 자체는 외부 비동기 — 표준 구조.
8. 검증 증거	TEST 6/7 (reset 중 CS_N=1, FIFO empty 등), TEST 10 (reset 후 레지스터).
9. 알려진 문제·한계	reset 폭이 1 clk 미만이면 동작은 설계 가정 밖. TEST 7 의 짧은 reset(약 14 ns)은 1–2 ACLK 폭 추정 (3.3절).
10. 합성 관점	ARESETn 는 SDC 에서 `set_false_path -from` 처리 확인 (SYN SDC).

2.8 sclk_gen · RTL-F 1256–1293

1. 목적	ACLK 를 4분주(2 ACLK 마다 toggle)해 SCLK 원본 `o_sclk` 생성.
2. 인터페이스	in: `clk`, `rst_n`. out: `o_sclk`.
3. 클럭 / 리셋	ACLK, async reset (RTL-007 후속). 리셋 중 `o_sclk`=0 으로 고정 → **SCLK 정지**.
4. 내부 구조	`r_toggle_phase` 1-bit 분주 phase + `o_sclk` 레지스터 (flop 출력이 곧 클럭 → BUFG).
5. 상태·주요 레지스터	`r_toggle_phase <= ~r_toggle_phase` 매 ACLK, phase=1 일 때 `o_sclk <= ~o_sclk`.
6. 타이밍 동작	o_sclk 주기 = 4 ACLK = 40 ns (25 MHz). 리셋 해제 직후 free-running (CS_N=1 중에도 toggle) — 그림 4-4 에서 관찰 확인.
7. CDC 관련	ACLK→SCLK 는 같은 소스에서 나온 generated clock. 데이터 CDC 와 별개로 STA 에서 generated clock 정의가 필요.
8. 검증 증거	LOG-13 TX_DEBUG 의 40 ns 샘플 간격, 그림 4-4/4-5 의 파형.
9. 알려진 문제·한계	외부 SPI clock 입력이 없고 SCLK 가 항상 구동되므로 **실제 SPI slave(외부 master)로 쓰려면 구조 변경 필요** — 명세(독립 SCLK)와의 불일치.
10. 합성 관점	Genus SDC: `create_generated_clock -divide_by 4` 확인. flop 출력이 클럭으로 쓰이는 경로는 lint 에서 “o_sclk_reg/d driven by a clock” 로 보고됨 확인 (01_check_timing_intent).

3 Clock / Reset / CDC 분석

그림 3-1 Clock / Reset 분배 구조 (RTL 코드에서 읽은 연결)



[확인] 연결 관계 = 최종 RTL(spi_bridge_top.v) 코드. [확인] 리셋 중 CS_N high 복귀 = LOG-13 TEST6/TEST7 PASS 줄.
[미확인] 비동기 assert 시 SCLK 글리치/불완전 펄스 영향, reset 해제 recovery/removal timing: 정적 RDC/STA 미수행.
[예정] reset_sync 출력 fanout, async reset 트리 timing (post-synthesis STA / RDC check).

3.1 클럭 도메인

도메인	주파수	소스	사용 블록
ACLK	100 MHz (TB 주기 10 ns)	외부 입력	register_block, sclk_gen, async_fifo(RX rd / TX wr), busy 2FF, pulse_sync(dst)
SCLK	25 MHz = ACLK ÷ 4	`sclk_gen` → BUFG (generated)	spi_master, spi_slave, async_fifo(RX wr / TX rd), pulse_sync(src), reset_sync_sclk

3.2 SCLK 는 독립 클럭인가, 생성 클럭인가

결론 확인: 최종 RTL 의 SCLK 는 **ACLK 에서 생성된 free-running generated clock** 입니다. 명세(SPEC)가 말하는 "독립 25 MHz 클럭"과 다릅니다. 이 차이는 (i) CDC 해석, (ii) SDC(generated clock 선언), (iii) 리셋 구조(SCLK 가 리셋 중 정지)에 영향을 줍니다. SCLK 가 생성 클럭이라고 해서 FIFO·펄스 경로의 동기기가 불필요해지는 것은 아닙니다 — 4:1 위상 관계가 PVT 와 배선 지연에서 보장되지 않기 때문에 설계는 두 도메인을 비동기로 취급하고 있으며(Gray + 2FF), 이는 보수적이고 타당한 선택으로 보입니다 추정.

3.3 Reset 구조와 RDC

- `ARESETn`(외부, 비동기) → `reset_sync` 2개. assert 는 즉시 두 도메인에 전파, release 는 각 도메인 클럭에 동기 확인.
- `sclk_gen` 은 `aclk_rst_n` 으로 reset 되므로 리셋 중 SCLK 가 정지한다. SCLK 도메인 flop 이 **동기 reset** 이면 리셋 중에는 클럭이 없어 reset 이 적용되지 않는다 — 이것이 RTL-007 의 원인(로그 RUN9: `FAIL: TEST6 CS inactive in reset expected=1 got=0`)이며, 수정은 SCLK 도메인 flop 을 `or negedge rst_n` 로 바꾸는 것이었다 확인.
- release 순서: ARESETn 해제 → aclk_rst_n(2 ACLK) → sclk_gen 시작 → SCLK 2 edge 후 sclk_rst_n 해제. 따라서 **ACLK 쪽 FIFO 읽기측이 SCLK 쪽 쓰기측보다 먼저 reset 에서 풀린다**. FIFO 는 읽기측에서 empty 로 시작하므로 기능적으로 안전하지만, 정식 RDC 분석은 수행하지 않았다 미수행.
- TEST 7 의 reset 폭: TB 주석은 7/13/55 ns 이나, 코드가 다음 ACLK negedge +1 ns 에서 해제하므로 실제 low 폭은 약 14/24/64 ns (ACLK negedge 가 10 ns 배수라는 가정) 추정. 로그의 `RESET #n asserted` 시각이 34,777/36,067/37,357 ns(1 ns 오프셋)인 것은 일치한다 확인.

3.4 CDC crossing 목록

ID	신호	방향	메커니즘 (RTL)	기능 시뮬레이션에서 확인된 것	정적 CDC 필요
X1	RX FIFO wr Gray ptr	SCLK→ACLK	2FF (`r_wr_ptr_gray_rclk`)	TEST 1/4 (top RX), TEST 8/9 (별도 인스턴스 Gray 단일 비트 전이 checker)	구조·reconvergence 검사 미수행
X2	RX FIFO rd Gray ptr	ACLK→SCLK	2FF → full 판정	TEST 4 FULL=1, level=16	동상
X3	TX FIFO wr/rd Gray ptr	ACLK↔SCLK	2FF 양방향	TEST 2/3/5 (TX burst, full 경계)	동상
X4	overflow	SCLK→ACLK	`pulse_sync` (toggle+2FF+XOR)	TEST 4 sticky/출력, TEST 6/7 reset 체크	동상
X5	spi_busy	SCLK→ACLK	**조합 OR** 후 2FF (lines 206–217)	STATUS[2] 는 어떤 TEST 도 직접 검사하지 않음 확인	**CDC-02** 열림
X6	rx_full	SCLK→ACLK	**동기기 없음** (STATUS[1]에 직접)	TEST 4 에서 FULL=1 이 읽혔으나 시뮬레이션은 metastability 를 모델링하지 않음	**CDC-01** 열림
X7	ARESETn	async→ACLK/SCLK	`reset_sync` ×2	TEST 6/7/10	RDC 분석 미수행

> **기능 시뮬레이션이 증명하는 것과 증명하지 못하는 것.** RTL 시뮬레이션은 metastability 와 타이밍을 모델링하지 않으므로, “PASS” 는 **논리적으로 올바른 순서·값**을 뜻할 뿐 CDC 안전성을 뜻하지 않습니다. Gray 포인터 단일 비트 전이는 TEST 8 의 checker 가 시뮬레이션 상 확인했으나, 이것은 **별도 FIFO 인스턴스**이며 top 안의 FIFO 에 대한 정적 CDC 증명이 아닙니다.

Gray 코드 변환표와 full 조건 계산은 6장 그림 6-3 참조.

3.5 열린 CDC 이슈

ID	내용	위험	권고 수정 (미적용)
CDC-01	`u_rx_fifo/o_full` (SCLK 도메인 register)가 `register_block` 의 STATUS[1] 로 동기기 없이 입력	ACLK flop 이 SCLK 천이 근처에서 샘플 → 메타스터블 가능. STATUS 읽기는 폴링 용도라 기능 영향은 낮지만 정식 CDC 도구는 위반으로 보고함 추정	`o_full` 을 ACLK 에서 2FF 로 동기하거나, rd 측 Gray 포인터로 ACLK 에서 full 을 재계산
CDC-02	`w_spi_busy = master_busy || slave_busy` (SCLK 도메인 조합) → ACLK 2FF	조합 출력 glitch 가 동기기에 들어갈 수 있음 추정	OR 결과를 SCLK 에서 register 한 뒤 2FF
—	`sclk_gen` flop 출력이 BUFG 입력으로 쓰이는 구조	STA/Lint 에서 “clock driven by flop” 항목	generated clock 정의로 처리 (Genus SDC 가 이미 수행 확인)

3.6 합성(Genus) 증거 — 무엇이고 무엇이 아닌가

> **중요:** 아래 수치는 Genus 22.17-s071_1 (2026-10-20 06:09, library `giolib045`, PVT 0P9V 125C) 로 합성한 netlist 의 리포트입니다. 이 netlist 는 **최종 RTL 과 다른 리비전**입니다: (a) `u_tx_clear_req_sync`, `u_tx_clear_done_sync` pulse_sync 가 있음(최종 RTL 에는 없음), (b) SDC 는 `XILINX_BUFG` 미정의(ifdef 가드)를 가정하나 최종 RTL 은 BUFG 무조건 인스턴스, (c) TX write enable 이 조합(`w_tx_fifo_wr_en`)이나 최종 RTL 은 registered. 따라서 **최종 RTL 의 합성·STA 결과가 아니며**, 구조가 비슷한 설계의 참고 데이터로만 제시합니다.

항목	값 (SYN 리포트)	비고
SDC 클럭	ACLK 1.0 ns (1 GHz), SCLK = generated ÷4 (4 ns)	1 GHz 는 교수님 규칙 사용자 보고
Setup slack	ACLK +66.6 ps (final.rpt 67 ps), SCLK +314.8 ps, violating path 없음	syn_generic +313 → syn_map +109 → syn_opt +67 ps
Critical path	`i_bus_addr[0]` → 조합 `w_tx_fifo_wr_en` → `u_tx_fifo/o_full_reg`, data path 704 ps	현재 최종 RTL 에서는 write enable 이 registered 라 경로 구조가 달라짐

항목	값 (SYN 리포트)	비고
면적	cell area 3,531.15, leaf 834, sequential 360	최종 RTL 의 값 아님
면적 분포	u_rx_fifo 2,179.5 / u_tx_fifo 720.8 / u_spi_slave 501.3 / u_spi_master 460.7 / u_register_block 344.4	
전력	total ≈ 1.05 mW	활동도 가정 미확인 미확인
Lint 요약	8건: loop-breaking cell 없음, multi-driver net 0, ARESETn 에 clocked external delay 없음, dbg 출력 external delay 없음, `o_sclk_reg/d` clock 구동	01_check_timing_intent
CDC 제약	`set_max_delay -datapath_only` 가 TUI-64 로 비활성	[TO VERIFY] 표기 확인

4 Debugging History

이 장은 보고서의 중심입니다. 증거 묶음에서 확인되는 오류만 기록했고, 로그·스크린샷·코드 diff 에 근거가 없는 오류는 만들지 않았습니다. “원인” 열이 **추정**인 곳은 실제로 원인을 보여주는 직접 증거(내부 신호 파형 등)가 없는 경우입니다.

4.1 Debugging History Table

ID	모듈	증상	원인	수정	상태	증거
RTL-001	async_fifo	full/empty 가 다음 포인터로부터 조합 생성 → 포인터 next 가 다시 `!o_full` 에 의존 (조합 루프)	`assign o_full = (w_wr_ptr_gray_next == …)`	registered `r_full` / `r_empty`	해결	코드 diff 확인; 증상 연결 추정
RTL-002	async_fifo	full 비교식에서 Gray 포인터 상위 2비트를 `~` (산술 부호반전)로 처리	`{~r_rd_ptr_gray_wclk[AW:AW-1], …}`	`~` (비트 반전)	해결	코드 diff 확인
RTL-003	async_fifo	쓰기 포인터가 읽기 도메인으로 전달되지 않음(상수 0). `o_empty` 영구 1, `o_rd_data`=xx, TIMEOUT	동기기 else 분기가 `<= 0`	2FF 로 `r_wr_ptr_gray` 전달	해결	SimVision 파형 + 콘솔 확인; 인과 추정
RTL-004	spi_master	`else if (state == S_SHIFT) ;` 의 stray semicolon → `o_mosi` 갱신이 조건 밖으로 이탈	세미콜론	세미콜론 제거	해결	코드 diff 확인; top 영향 없음
RTL-005	spi_master	`o_rx_captured_valid` 가 SHIFT 아닌 모든 cycle 에서 1	`end else begin`	`else if (state == S_DONE)`	해결	코드 diff 확인; 출력 미연결
RTL-006	spi_slave	TX burst 0x11/0x22/0x33 인데 MISO 가 0x33 만 16 프레임 반복, `FAIL: TX burst incomplete (0/3)`	`w_tx_pop = !empty && state != S_RECEIVE` 가 매 cycle pop → latch 덮어쓰기	hold register + consume 로직	해결	LOG-100015 / 121810 / LOG-13 확인
RTL-007	SCLK 도메인 전체	reset 중 `CS_N=0` 유지 (`FAIL: TEST6 CS inactive in reset expected=1 got=0`)	SCLK 가 리셋 중 정지, flop 은 sync reset	SCLK 도메인 flop 을 `or negedge rst_n`	해결	LOG-13 RUN9→RUN10 확인
RTL-008	register_block	FIFO 에 2개 이상 쌓이면 RX 1 byte 손실 `expected 01 got 02`	`w_rx_prefetch` 에 `!r_rx_fifo_rd_en_d` 없음 → 연속 2 pop	미수정	미해결	LOG-093905 + 코드 확인
TB-001	tb_register_block	TX_DATA 쓰기 후 `o_tx_fifo_wr_en pulsed expected=1 got=0`, `wr_data 0xC3 expected got=0`	TB 의 타이밍 가정과 DUT 의 불일치 (원인 미확정)	—	미확정	SS-00 확인; 원인 추정
TB-002	tb_async_fifo	`PASS: o_empty asserted after draining 16 entries` 가 통과했는데 TEST 3 은 TIMEOUT 200 µs	`o_empty` 가 항상 1 이라 “empty 확인” 이 공허 (vacuous)	TB 에 데이터 비교 필요 / RTL-003 수정	일부 해결	SS-02 콘솔 확인
TB-003	tb top (초기)	MISO 재구성 `expected=5a got=0` → `got=17` (5,345 ns), TX_DATA_REG→MISO 실패 (8,526 ns)	TX 경로 미완성 + TB capture 타이밍 (원인 미분리)	TX hold + TB 보정	해결 (RUN5)	누적 로그 확인; 분리 추정
TB-004	TB TEST 7	`FAIL: TEST7 RX valid reset expected=0 got=1` ×3	reset assert 후 1 ns 시점 검사: `register_block` 은 sync reset 이라 다음 ACLK edge 전에는 old 값 유지	검사를 55 ns reset 펄스에만 적용	해결 (RUN12)	LOG-13 확인; 원인 추정

ID	모듈	증상	원인	수정	상태	증거
TB-005	TB TEST 7	주석의 reset 폭(7/13/55 ns)이 실제 코드 동작(≈14/24/64 ns)과 불일치	주석 오류	미수정 (본 보고서에 기록)	문서화	TB 코드 분석 추정

> **해석 주의**: RTL-001~003 은 Cadence 시대의 초기 RTL(`spi_bridge_all_in_one.v` 계열)에 있었고, 수정 후 파일(`…_fixed.v`)과의 diff 가 증거입니다. 어떤 시뮬레이션 run 이 정확히 어느 리비전을 사용했는지는 증거 묶음이 보장하지 않으므로, "증상 ↔ 원인" 연결은 코드 구조와 파형이 서로 모순되지 않는다는 수준(추정)으로만 기술합니다.

4.2 상세 분석

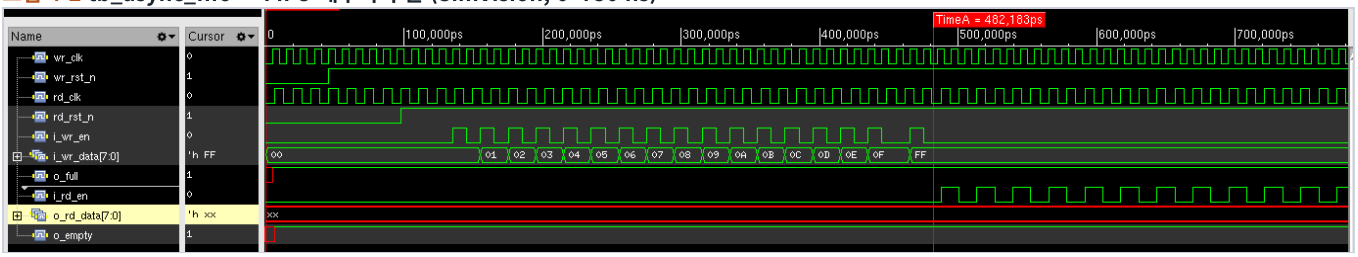
RTL-001 / 002 / 003 async_fifo bring-up — 조합 루프, 연산자 오타, 포인터 동기 상수 0	4.2-A · 해결
Problem	`tb_async_fifo` 에서 FIFO 에 16개를 쓰고 읽어도 `o_rd_data` 는 계속 `xx`, `o_empty` 는 1 로 고정되어 TEST 3(동시 읽기/쓰기)이 끝나지 않았다.
Error Evidence	SimVision 콘솔 (SS-02, 10/14 04:20): `[455000] PASS: o_full asserted after 16 writes` → `[904000] PASS: o_empty asserted after draining 16 entries` → `[200000000] FAIL: TIMEOUT -- simulation did not finish` → `xmsim: *E,RNFNSH: Cannot continue simulation due to a previous $finish`. 파형(그림 4-2, 4-3)에서 `i_rd_en` 이 토글해도 `o_rd_data` 는 xx, `o_empty` 는 1.
Root Cause	세 결함이 겹쳐 있었다. ① `o_full`/`o_empty` 를 `assign` 으로 "다음 포인터" 와 비교하는데, 다음 포인터는 다시 `!o_full`/`!o_empty` 에 의존 → **조합 루프**. ② full 비교에서 `-r_rd_ptr_gray_wclk[…]` 를 써 2-bit 값이 비트 반전이 아니라 산술 부호반전이 됨. ③ 읽기 도메인의 쓰기-포인터 동기기 else 분기가 상수 0 → 읽기 쪽은 쓰기를 전혀 보지 못해 `empty` 가 풀리지 않음. 직접적인 "empty 고정"은 ③, TEST 3 timeout 은 ③의 결과로 보임 추정.

Before Fix	`// spi_bridge_all_in_one.v (L751-752, L783-784, L790)`
	`assign o_full = (w_wr_ptr_gray_next == {-r_rd_ptr_gray_wclk[ADDR_WIDTH:ADDR_WIDTH-1],`
	`r_rd_ptr_gray_wclk[ADDR_WIDTH-2:0]});`
	`... r_wr_ptr_gray_meta_r <= {(ADDR_WIDTH+1){1'b0}}; // 동기기가 상수 0`
	`r_wr_ptr_gray_rclk <= {(ADDR_WIDTH+1){1'b0}};`
	`assign o_empty = (w_rd_ptr_gray_next == r_wr_ptr_gray_rclk);`

After Fix	`// spi_bridge_all_in_one_fixed.v → 최종 RTL 에도 동일 (RTL-F L874, L912)`
	`r_full <= (w_wr_ptr_gray_next == {~r_rd_ptr_gray_wclk[AW:AW-1], r_rd_ptr_gray_wclk[AW-2:0]});`
	`r_empty <= (w_rd_ptr_gray_next == r_wr_ptr_gray_rclk);`
	`r_wr_ptr_gray_meta_r <= r_wr_ptr_gray; r_wr_ptr_gray_rclk <= r_wr_ptr_gray_meta_r;`

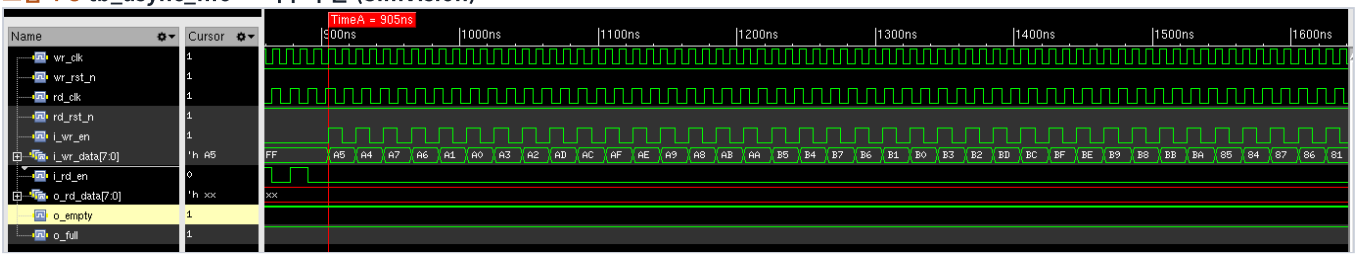
Engineering Explanation	① 조합 루프는 시뮬레이션에서 X 가 고착되고, 합성에서는 loop-breaking 이 필요한 구조적 오류다. flag 를 register 로 만들면 한 cycle 지연되지만 루프가 끊긴다 — 이 지연은 FIFO 에 보수적(full 은 늦게 풀리고 empty 는 늦게 풀림)으로만 작용한다. ② Gray full 판정은 상위 2비트 반전 + 나머지 동일이 정의이므로 `~` 가 필수. ③ 동기기에 상수를 넣으면 "시뮬레이션은 돌지만 의미가 없는" 회로가 되어 TB 가 vacuous PASS(TB-002)를 만들 수 있다.
Verification After Fix	Vivado 파형(그림 4-6 → 4-7): 수정 전으로 추정되는 캡처에서는 `r_wr_ptr_bin/gray` 가 800 ns 이후 XX, `o_full/o_empty` X. 수정 후로 추정되는 캡처에서는 두 포인터가 00→06 으로 lockstep 증가, `r_full`=0, `o_rd_level`=00 추정 (캡처 순서는 파일명 시각 기준). 최종 TB: TEST 8 (16 write → full, 16 read → empty, wrap, Gray 단일 비트 전이), TEST 9 (128 byte) 모두 PASS 확인.
Final Status	**해결.** 단 TEST 8/9 가 검사한 FIFO 는 TB 가 만든 별도 인스턴스이며, top 내부 FIFO 의 정적 CDC 검증은 **미수행**.

그림 4-2 tb_async_fifo — FIFO 채우기 구간 (SimVision, 0–750 ns)



관련	RTL-003, TB-002
주요 신호	`wr_clk, rd_clk, i_wr_en, i_wr_data[7:0], o_full, i_rd_en, o_rd_data[7:0], o_empty`
관찰	`i_wr_data` 가 01→0F→FF 로 쓰이고 약 500 ns 부터 `i_rd_en` 이 토글하지만 `o_rd_data` 는 `'h xx` 로 고정되고 `o_empty`=1 확인
의미	읽기 도메인이 쓰기를 보지 못하는 증상. RTL-003 과 일치 추정. 시각 단위 ps, 커서 482,183 ps

그림 4-3 tb_async_fifo — 이후 구간 (SimVision)



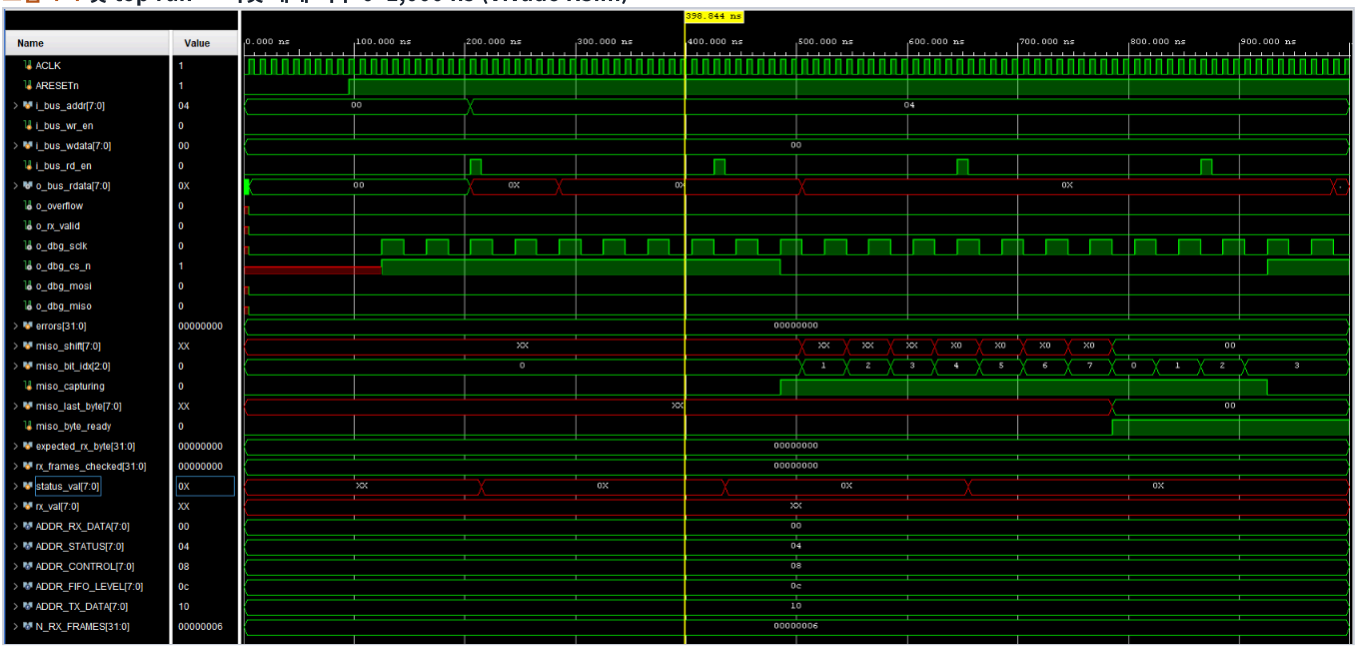
관련	RTL-003
관찰	읽기 enable 이 반복되어도 `o_rd_data` 가 xx 인 상태 지속 확인
의미	TB-002: 이 상태에서도 “drain 후 o_empty 확인” 이 PASS 할 수 있어 TB 가 FIFO 결함을 놓침

SIM-X-1 첫 top 레벨 Vivado 실행 — STATUS 가 X, RX 프레임 0/6 체크 후 TIMEOUT

4.2-B · 원인 부분 확인

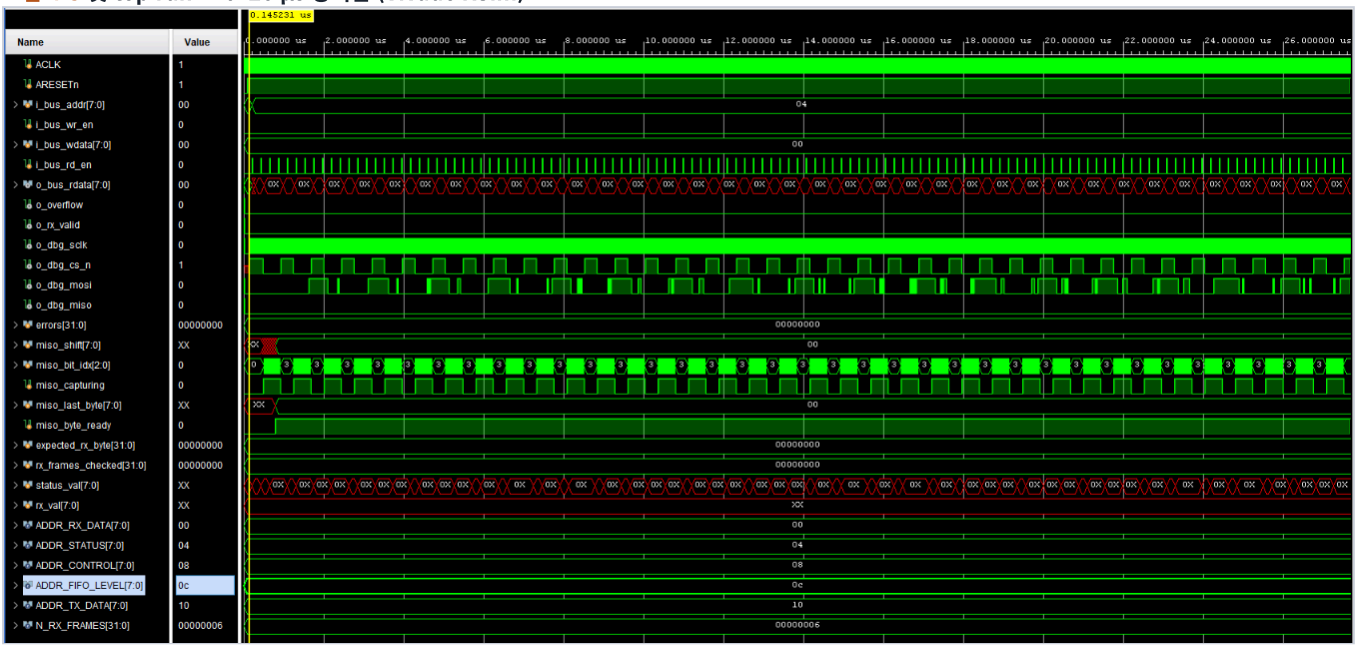
Problem	top 레벨 TB 를 Vivado 에서 처음 돌렸을 때 `o_bus_rdata` 가 `0X` 로 읽히고 RX 프레임이 하나도 검사되지 못했다.
Error Evidence	누적 Vivado 로그: `[2000000000] FAIL: TIMEOUT -- only 0/6 RX frames checked` (2 ms). 그림 4-4: ARESETn 해제(≈100 ns) 후 `o_dbg_sclk` 가 CS_N=1 인 동안에도 toggle, `i_bus_rd_en` 펄스마다 `o_bus_rdata`=`0X`, `status_val`=XX/0X, `miso_shift`=XX.
Root Cause	STATUS 하위 4비트가 X — FIFO flag(empty/full)와 pointer 가 X 인 상태(그림 4-6)와 일치. 직접 신호 추적으로 확정한 기록은 없으므로 **FIFO 조합 루프/초기화 결함(RTL-001~003)에 의한 X 전파로 추정** 추정.
Before Fix	RTL-001~003 미수정 상태(추정).
After Fix	RTL-001~003 의 수정. 이후 같은 TB 계열에서 RX 시퀀스 확인이 가능해짐.
Engineering Explanation	X 가 레지스터 read mux 로 전파되면 TB 의 `===` 비교는 실패하지만 폴링 루프(`while (status[3]==0)`)는 X 를 0 으로 취급하지 못해 무한 대기 → timeout 으로 나타난다. 이 때문에 최종 TB 는 모든 대기 루프에 timeout 을 두었다(`#2000000`, TB-F L1074).
Verification After Fix	LOG-13 TEST 1: RX_DATA 가 0,1,2,3,4,5 로 읽혀 6 PASS (1,115–4,835 ns) 확인.
Final Status	해결 (증상 소멸 확인). 원인 분리는 추정.

그림 4-4 첫 top run — 리셋 해제 직후 0–1,000 ns (Vivado XSim)



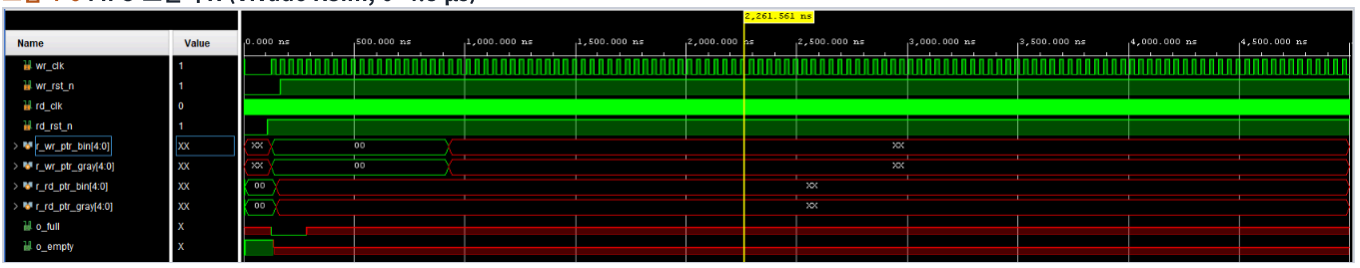
관련	SIM-X-1, SCLK free-running
주요 신호	`ARESETn, i_bus_addr, i_bus_rd_en, o_bus_rdata, o_dbg_sclk, o_dbg_cs_n, miso_shift, status_val`
관찰	ARESETn=1 이 ≈100 ns. `o_dbg_sclk` 는 그 직후부터 **CS_N=1 인 상태에서도** 40 ns 주기로 toggle. 읽기 펄스(≈200, 400, 630, 850 ns) 후 `o_bus_rdata`=`0X`. CS_N 이 ≈480 ns 에 0 으로.
의미	① SCLK 가 free-running 임을 파형으로 확인 **확인**. ② STATUS 의 하위 nibble 이 X — 위 원인 추정의 근거

그림 4-5 첫 top run — 0–26 µs 장시간 (Vivado XSim)



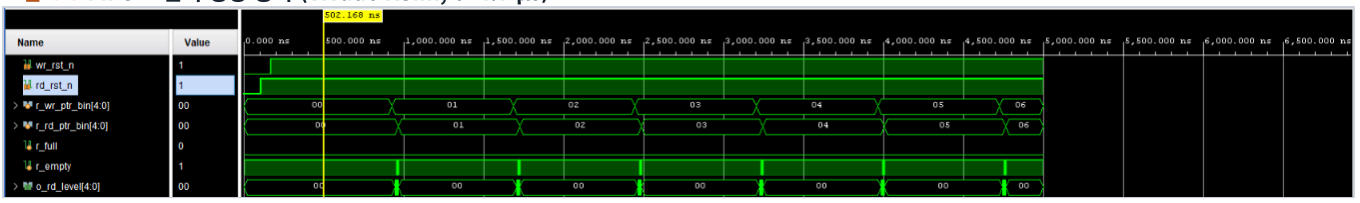
관련	SIM-X-1
관찰	26 µs 동안 `o_bus_rdata` 가 계속 0X, `status_val` X 반복, `rx_frames_checked`=0 **확인**
의미	TB 가 STATUS[3](RX_VALID) 를 기다리다 영원히 진행하지 못함 — timeout 의 직접 원인

그림 4-6 FIFO 포인터 X (Vivado XSim, 0–4.8 µs)



관련	RTL-001/003
주요 신호	`wr_clk`, `wr_rst_n`, `rd_clk`, `rd_rst_n`, `r_wr_ptr_bin/gray`, `r_rd_ptr_bin/gray`, `o_full`, `o_empty`
관찰	리셋 직후 포인터=00, ≈800 ns 이후 `r_wr_ptr_*` 가 XX 로, `r_rd_ptr_*`, `o_full`, `o_empty` 도 X. 이 캡처에서 `rd_clk` 신호값=0, 파형은 연속 toggle(압축) 확인
의미	포인터 자체가 X — 조합 루프/X 전파 징후 추정. 리비전은 불명

그림 4-7 FIFO 포인터 정상 동작 (Vivado XSim, 0–4.7 µs)



관련	RTL-001/003 수정 후
관찰	`r_wr_ptr_bin`, `r_rd_ptr_bin` 이 00→06 으로 같은 간격으로 증가, `r_full`=0, `r_empty` 는 대부분 1 이고 포인터 증가 시점마다 짧은 변화(해상도 한계로 세부 판독 불가), `o_rd_level`=00 확인
의미	쓰기 포인터가 읽기 측에 전달되고(읽기가 따라감) X 가 없음. 단 이 캡처는 write/read 가 번갈아 일어나는 시나리오로 보이며 full 경계는 포함하지 않음 추정.

RTL-006 spi_slave TX 경로 — FIFO 에서 여러 바이트를 한꺼번에 pop 하여 마지막 바이트(0x33)만 반복 4.2-C · 해결

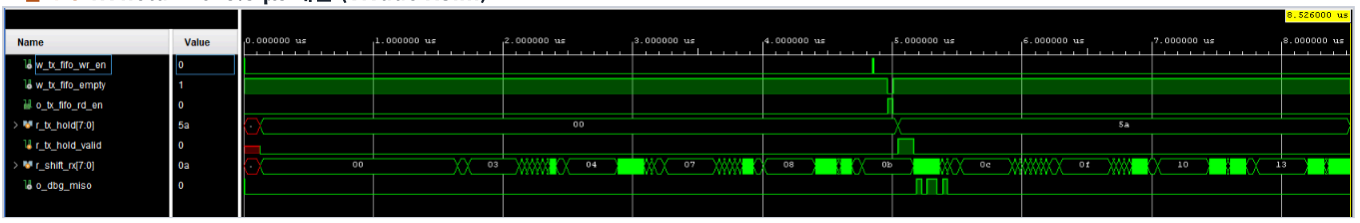
Problem	TX 로 0x11, 0x22, 0x33 을 순서대로 쓰고 MISO 를 관찰했더니 세 프레임이 각각 0x11/0x22/0x33 이어야 하는데, 0x33 이 계속 반복되었다.
Error Evidence	`spi_tx_burst` TB (LOG-100015): TX FIFO 가 0x11/0x22/0x33 을 965/985/1005 ns 에 수락 → `MISO FRAME 1…16 = 0x33` 연속 → `FAIL: TX burst incomplete (0/3)` @13,055 ns.
Root Cause	이전 slave 는 `w_tx_pop = !i_tx_fifo_empty && (state != S_RECEIVE)` 로, RECEIVE 가 아닌 한 **매 SCLK 마다 FIFO 를 pop**했다. 프레임 사이에 3개가 연속으로 pop 되면서 단일 래치(`r_tx_byte_latched`)가 덮어씌워져 마지막 값 0x33 만 남았고, FIFO 가 비면 래치가 그대로 유지되어 이후 모든 프레임이 0x33 이 되었다 (코드 구조와 로그 일치).

Before Fix	`// spi_bridge_all_in_one_fixed.v  L604-627`
	`wire w_tx_pop = !i_tx_fifo_empty && (state != S_RECEIVE);`
	`assign o_tx_fifo_rd_en = w_tx_pop;`
	`... if (r_tx_pop_d) r_tx_byte_latched <= i_tx_fifo_rd_data;`
	`... r_shift_tx <= r_tx_byte_latched;  // 프레임 시작 시 항상 최신 래치`

After Fix	`// 최종 RTL-F (spi_slave)`
	`assign w_tx_pop = !i_tx_fifo_empty && !r_tx_hold_valid && !r_tx_pop_d;`
	`assign w_tx_consume = (state==S_RECEIVE) && (bit_cnt==3'd0) && !i_cs_n && r_tx_cur_from_hold;`
	`if (r_tx_pop_d)  begin r_tx_hold <= i_tx_fifo_rd_data; r_tx_hold_valid <= 1'b1; end`
	`if (w_tx_consume)  r_tx_hold_valid <= 1'b0;  // 실제로 전송을 시작할 때만 해제`
	`// 프레임 시작: hold 가 valid 면 로드, 아니면 0x00 을 보냄`

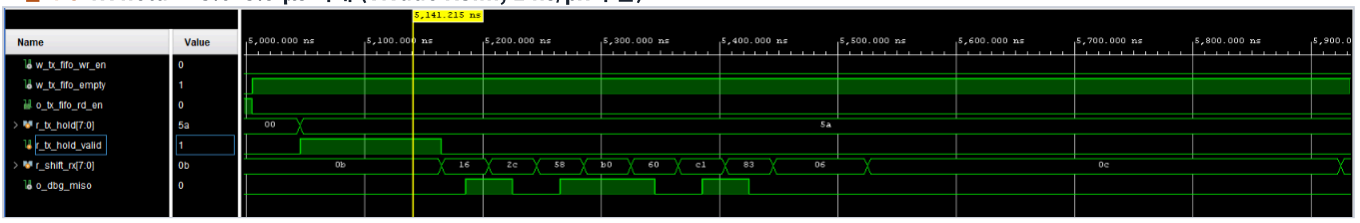
Engineering Explanation	FIFO 읽기는 “한 번에 하나, 소비될 때까지 대기” 여야 한다. `hold_valid` 는 hold 가 비어 있을 때만 새 pop 을 허용(`!r_tx_hold_valid`)하고, `r_tx_pop_d` 는 FIFO 출력이 1 cycle 늦게 유효한 것에 맞춰 중복 pop 을 막는다. `w_tx_consume` 는 “이 바이트가 실제로 shift register 로 간 바로 그 프레임” 에서만 hold 를 비운다. TX FIFO 가 비어 있으면 0x00 이 나가며, 이는 **정의된 동작**이지만 명세에 근거한 것인지는 미확인 미확인.
Verification After Fix	① LOG-121810 (수정 RTL): MISO `FRAME 1=0x11 @1655 ns, 2=0x22 @2415 ns, 3=0x33 @3175 ns` → `ALL TESTS PASSED (spi_tx_burst)`. ② LOG-13 TEST 2: `PASS: TX_DATA_REG to MISO = 1` (0x5A, 5,486 ns), TEST 3: 0x11 @6,246, 0x22 @7,006, 0x33 @7,766 ns, `PASS: TX Burst sequence 11 -> 22 -> 33` 확인. ③ 파형: 그림 4-8 ~ 4-10, 7-2, 7-3.
Final Status	**해결.** 한계: 3-byte burst 만 검증. TX FIFO 가 프레임 도중 채워지는 경우, 연속 16 프레임 등은 미검증.

그림 4-8 TX hold — 0–8.5 µs 개관 (Vivado XSim)



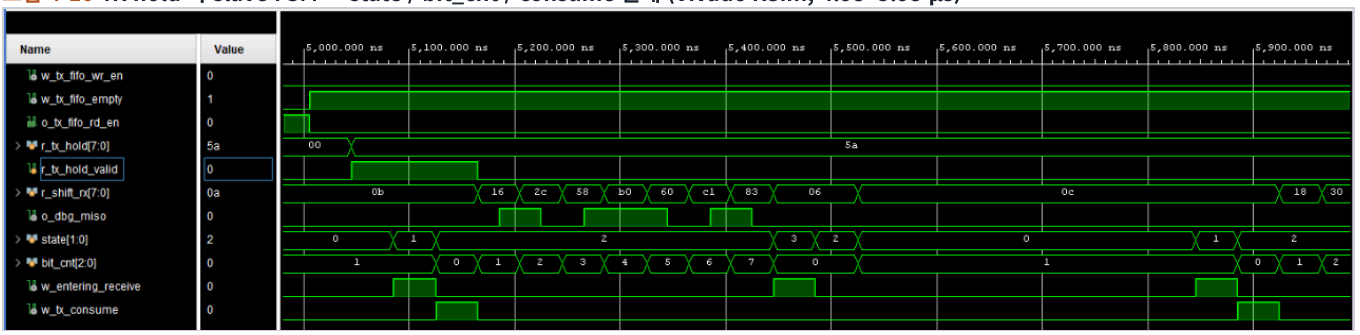
관련	RTL-006 After Fix
주요 신호	`w_tx_fifo_wr_en, w_tx_fifo_empty, o_tx_fifo_rd_en, r_tx_hold[7:0], r_tx_hold_valid, r_shift_rx[7:0], o_dbg_miso`
관찰	≈4.7 µs 에 `w_tx_fifo_wr_en` 1 cycle, `w_tx_fifo_empty` 1→0, 그 직후 `o_tx_fifo_rd_en` 펄스, `r_tx_hold`: 00→5a, `r_tx_hold_valid` 펄스 → ≈5.2–5.4 µs 에 `o_dbg_miso` 가 세 번 high. 커서 8,526 ns.
의미	쓰기 → pop → hold → MISO 의 한 번에 하나 흐름이 파형에서 확인됨 확인.

그림 4-9 TX hold — 5.0–5.9 µs 확대 (Vivado XSim, 1 ns/px 수준)



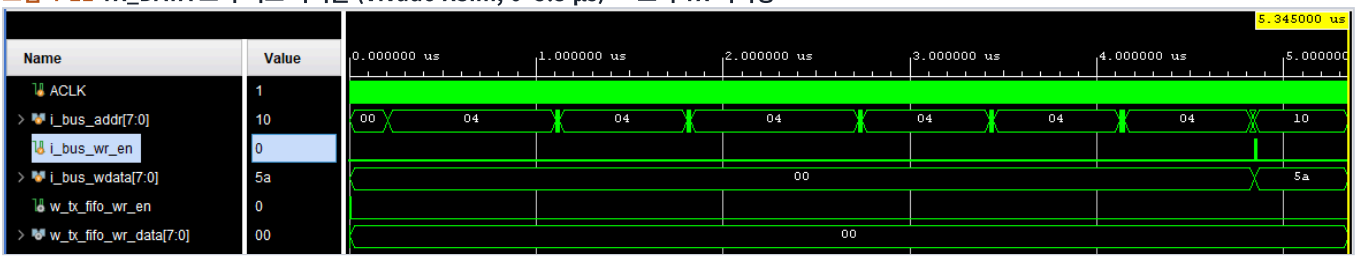
관련	RTL-006
관찰	`r_tx_hold`=5a, `r_tx_hold_valid`=1 이 약 5,045–5,165 ns. `o_dbg_miso` high 구간이 약 5,170–5,210, 5,250–5,330, 5,370–5,410 ns → 0x5A = 0101 1010 의 high 비트(bit6, bit4·3, bit1)와 일치 확인
의미	0x5A 가 직렬로 출력됨. 단 0x5A(0101 1010)는 비트 순서를 뒤집어도 같은 값(대칭)이라 **이 파형만으로는 MSB/LSB 순서를 증명하지 못한다** — 순서는 비대칭 값인 TEST 3 의 0x11/0x22/0x33 이 뒷받침한다. (high 구간 경계는 눈금에서 읽은 근사값)

그림 4-10 TX hold 와 slave FSM — state / bit_cnt / consume 관계 (Vivado XSim, 4.95–5.95 µs)



관련	RTL-006
주요 신호	`state[1:0]`, `bit_cnt[2:0]`, `w_entering_receive`, `w_tx_consume`, `r_tx_hold_valid`
관찰	`w_entering_receive` 가 LOAD→RECEIVE 직전, `state` 0→1→2, `bit_cnt` 0→7, `w_tx_consume` 펄스는 `bit_cnt=0` 의 첫 RECEIVE cycle(≈5,085–5,125 ns)에 발생하고 이때 `r_tx_hold_valid` 가 내려감. 프레임 종료 후 state 3→2→0.
의미	hold 는 “소비되는 프레임의 첫 비트” 에서만 해제됨 — 설계 의도(Process 5/6)와 일치 확인.

그림 4-11 TX_DATA 쓰기 버스 사이클 (Vivado XSim, 0–5.3 µs) — 초기 TX 디버깅



관련	TB-003
관찰	`i_bus_addr`: 04(상태 폴링) → 10, `i_bus_wr_en` 1 cycle(≈4.7 µs), `i_bus_wdata`=5a. 같은 캡처에서 `w_tx_fifo_wr_en` 은 0 으로 표시 (이 시간 해상도에서 펄스는 보이지 않음) 확인
의미	쓰기 입력 측만 확인됨. 이 시점의 로그는 `FAIL: _miso reconstructed byte == 0x5A expected=5a got=0` @5,345 ns (커서). 내부 경로가 어디서 끊겼는지는 이 캡처만으로 결정 불가 미확인. 이후 그림 4-8 에서 같은 쓰기가 TX FIFO 로 들어가는 것이 확인됨(리비전 차이 가능성).

RTL-007 SCLK 도메인 동기 reset — 리셋 중 SCLK 가 멈춰 CS_N 이 초기화되지 않음 4.2-D · 해결

Problem	TEST 6(프레임 중간 reset)에서 reset 이 걸렸는데도 `o_dbg_cs_n` 이 0 으로 남았다.
Error Evidence	LOG-13 RUN9: `[32610000] TEST6 ASSERT RESET mid-frame CS=0 BIT=3 → [32730000] FAIL: TEST 6 CS inactive in reset expected=1 got=0`. RUN10 (수정 후): `[32730000] PASS: TEST6 CS inactive in reset = 1`.
Root Cause	`sclk_gen` 은 `aclk_rst_n` 으로 reset 되므로 리셋 동안 `o_sclk` 가 0 으로 고정되어 SCLK 가 멈춘다. SCLK 도메인의 master/slave/FIFO/pulse_sync flop 이 **동기 reset**(`always @(posedge SCLK)` + `if (!rst_n)`)이면 클럭 edge 가 없어 reset 이 적용되지 않는다. 코드 diff: master·slave·pulse_sync·sclk_gen 의 always 블록 약 18 줄이 `or negedge rst_n` 로 바뀜.

Before Fix	`always @(posedge SCLK) begin  // sync reset: SCLK 정지 시 무의미`
	`  if (!rst_n) ... else ...`
	`end`

After Fix	`always @(posedge SCLK or negedge rst_n) begin  // async assert`
	`  if (!rst_n) ... else ...`
	`end`

Engineering Explanation	리셋을 담당하는 클럭이 리셋 자체로 멈추는 구조에서는 **비동기 assert**가 필수다. 반대로 release 는 `reset_sync` 가 SCLK 에 동기화해 주므로 recovery/removal 은 해당 도메인에서 정의된다. ACLK 도메인(`register_block`)은 항상 구동되므로 sync reset 이 허용되며, 이 때문에 TB-004 처럼 "reset assert 직후 1 ns 에 register_block 상태가 아직 안 변하는" 현상이 생긴다.
Verification After Fix	LOG-13: RUN10 의 TEST 6 PASS 7건(`CS inactive`, `RX FIFO empty`, `TX FIFO empty`, `overflow cleared`, `RX valid cleared`, `first RX byte after reset = 0`, `TX recovered after reset`), TEST 7 (3회 reset) PASS 17건 확인. 그림 6-2.
Final Status	**해결 (기능 시뮬레이션 기준).** RDC/정적 reset 분석은 **미수행**. async reset 사용으로 인해 reset 해제 시 recovery/removal 타이밍 확인이 STA 단계에서 필요 예정.

RTL-008 register_block RX prefetch — 연속 double-pop 으로 바이트 손실 (미해결)	4.2-E · 미해결
Problem	RX FIFO 에 2개 이상 쌓인 상태에서 소프트웨어가 순서대로 읽으면 1 바이트가 사라진다.
Error Evidence	LOG-093905 (queue-scoreboard TB): `[79280000] FAIL T04 RX backlog (queue scoreboard): LOSS of 1 byte(s): expected 01 got 02`. 같은 TB 에서 39/84 항목 FAIL.
Root Cause	`w_rx_prefetch = !rx_data_buf_valid && !i_rx_fifo_empty && !r_drain_active;` 에 "이미 pop 요청한 cycle 의 다음 cycle" 을 막는 조건(`!r_rx_fifo_rd_en_d`)이 없다. pop 직후 cycle 에는 `rx_data_buf_valid` 가 아직 0 이고 `empty` 도 (FIFO 에 2개 이상이면) 0 이므로 prefetch 가 한 번 더 발생한다. 첫 바이트가 buffer 에 래치되는 같은 시점에 두 번째 pop 의 결과가 다음 cycle 에 buffer 를 덮어쓴다 (`else if (r_rx_fifo_rd_en_d)` 가 읽기 소비보다 우선).
Before Fix	최종 RTL-F L1032 `w_rx_prefetch` 정의 그대로 (미수정).
After Fix	**미적용.** 제안: `w_rx_prefetch = !valid && !empty && !drain && !r_rx_fifo_rd_en_d;` 예정
Engineering Explanation	FIFO 읽기 데이터가 1 cycle 늦게 나오는 구조에서 "요청 중" 상태를 추적하지 않으면 중복 요청이 생긴다. TX 쪽(`r_tx_pop_d`)에서는 같은 문제를 이미 막았는데 RX 쪽에는 대칭 보호가 빠졌다.
Verification After Fix	해당 없음. **최종 TB(TEST 1–10)는 이 결함을 검출하지 못한다**: TEST 1 은 프레임마다 읽어 backlog 가 거의 없고, TEST 4 는 값 비교 없이 개수·flag 만 검사한다 확인(TB 코드 분석). 즉 "ALL TESTS PASSED" 가 "RX 경로가 완전하다"는 뜻이 아니다.
Final Status	**미해결 · 최종 RTL 에 존재.** 우선순위 높음. 수정 후 queue-scoreboard TB 재실행 필요 예정.

4.3 기타 이슈 요약

ID	내용	근거 / 판단
RTL-004	`else if (state == S_SHIFT) ;` 로 if 가 빈 문장이 되어 뒤따르는 `o_mosi <= r_shift_out[7]` 가 항상 실행 → reset 구문에서 벗어난 MOSI 갱신. 세미콜론 제거(L424).	diff 확인. top 에서는 MOSI 가 debug 출력이라 TEST 영향 없음
RTL-005	`o_rx_captured_valid` 가 SHIFT 이외 모든 상태에서 1 → `S_DONE` 에서만(L442).	diff 확인. 출력 미연결이라 TEST 로 검증 불가 미확인
TB-001	Cadence 콘솔 (SS-00): `[145000] FAIL: o_tx_fifo_wr_en pulsed expected=1 got=0`, `[145000] FAIL: o_tx_fifo_wr_data == 0xC3 expected=c3 got=0`, `[165000] FAIL: TX_DATA_REG readback == 0xC3 expected=c3 got=0`, `[205000] FAIL: EG unaffected by unrelated write expected=c3 got=0`. 직후 TEST 4/2 의 FULL·OVERFLOW 는 PASS.	TX_DATA 쓰기 경로 한정 실패. TB 의 타이밍 가정(같은 cycle 펄스)과 DUT 의 registered write enable 의 불일치 가능성이 있으나 원인 문서 없음 추정
TB-002	tb_async_fifo TEST 1/2 가 FIFO 결함 상태에서도 PASS (`o_full`, `o_empty` flag 만 확인, 데이터 미비교) → 공허한 PASS.	교훈: PASS 만으로 부족, 데이터 비교와 시나리오 내 비-자명한 상태 필요
TB-003	LOG(5)(7): `[5345000] FAIL: _miso reconstructed byte == 0x5A expected=5a got=0` → 이후 `got=17`, `[8526000] FAIL: TX_DATA_REG to MISO expected=1 got=0` (RUN1–4 동일). RUN5 부터 PASS.	RUN4→RUN5 사이 TX hold 도입(RTL-006)과 함께 해소. TB 문제와 RTL 문제의 분리는 증거 부족 추정

ID	내용	근거 / 판단
TB-004	RUN11 FAIL ×3 @34,778 / 36,068 / 37,358 ns → RUN12 에서 55 ns 펄스에만 검사. 3회 모두 `expected=0 got=1`.	위 RTL-007 설명 참조. TEST 7 이 “짧은 reset 에서 RX_VALID 가 즉시 0” 을 요구하는 것은 sync reset 구조와 모순 — 검증 목표 자체가 약해졌음을 의미 (8장)
TB-005	TB 주석(7/13/55 ns) vs 코드 동작(≈14/24/64 ns)	로그에서 reset assert 시각이 일치(34,777/36,067/37,357)하므로 코드 해석이 맞다고 판단 추정

5 검증 전략과 Testbench 구조

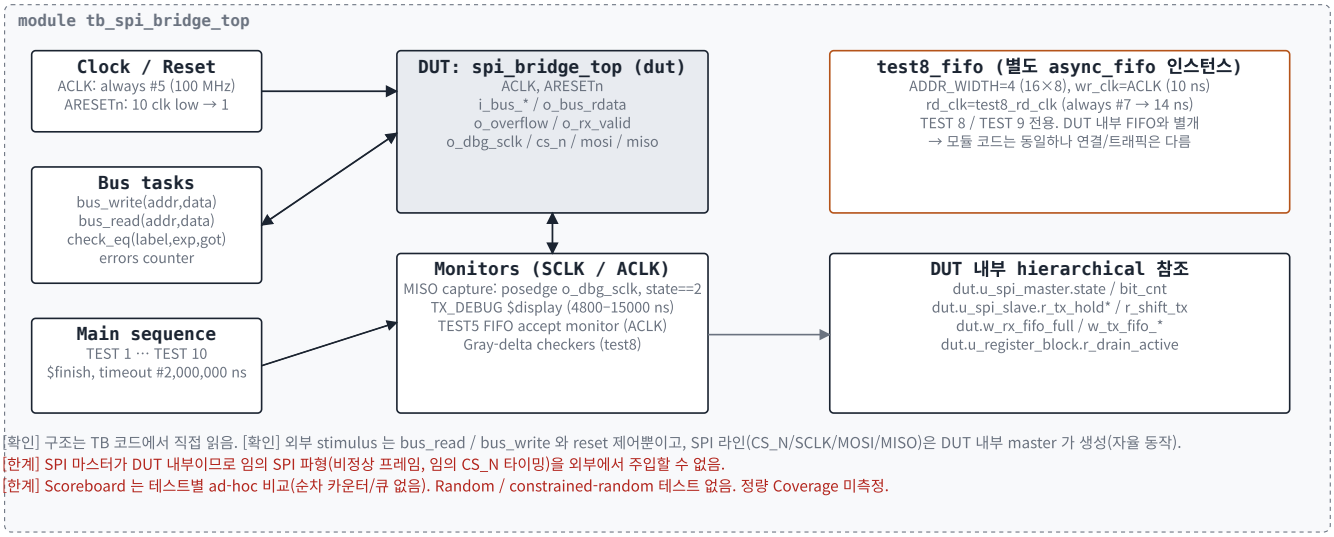
5.1 검증 목표

- **기능 정합성**: SPI 프레임이 RX FIFO 를 거쳐 레지스터로 전달되고(RX), 버스에서 쓴 바이트가 MISO 로 나가는지(TX).
- **경계 조건**: RX FIFO full/overflow, TX FIFO full(4개), 레지스터 corner case.
- **Reset 복구**: 프레임 도중, 반복 reset 이후 정상 동작 복귀.
- **CDC 구조의 기능적 정당성**: Gray 포인터 FIFO 의 full/empty/wrap/동시 접근 — **기능 시뮬레이션 범위**이며 metastability 나 타이밍은 대상이 아님.

5.2 환경

시뮬레이터	Vivado 2025.1 XSim (최종), 이전 Cadence Xcelium/SimVision
TB 파일	`tb_spi_bridge_top.v`, 1,090 lines, top `tb_spi_bridge_top`, 파일 위치(로그): `C:/Users/user/Desktop/KKKKK/vivado/SPI.srcs/sim_1/new/`
클럭	ACLK 10 ns 주기. TEST 8/9 용 `test8_rd_clk` 14 ns 주기 (ACLK 와 비정수비)
종료	모든 TEST 후 `errors==0` 이면 `ALL TESTS PASSED`, 아니면 `TESTS FAILED: n error(s)`. 전체 timeout `#2000000` (2 ms, L1074)
측정 방식	로그의 `[%0t]` 는 ps 단위: `[44896000]` = 44,896 ns

그림 5-1 최종 Testbench 구조 (tb_spi_bridge_top.v, 1090 lines)



[확인] 구조는 TB 코드에서 직접 읽음. [확인] 외부 stimulus 는 bus_read / bus_write 와 reset 제어뿐이고, SPI 라인(CS_N/SCLK/MOSI/MISO)은 DUT 내부 master 가 생성(자율 동작).
[한계] SPI 마스터가 DUT 내부이므로 임의 SPI 파형(비정상 프레임, 임의 CS_N 타이밍)을 외부에서 주입할 수 없음.
[한계] Scoreboard 는 테스트별 ad-hoc 비교(순차 카운터/큐 없음). Random / constrained-random 테스트 없음. 정량 Coverage 미측정.

5.3 Self-checking 방식

- `check_eq(label, expected, actual)`: 불일치면 `FAIL` 출력과 `errors++`. **주의:** label 인수가 `[255:0]` 폭이라 32자를 넘는 label 은 **앞부분이 잘려** 출력된다 (예: `EST7 CS released…`, `ST10 unmapped…`). 로그 가독성 문제일 뿐 판정에는 영향 없음 확인.
- MISO 모니터(L158–209): `o_dbg_miso` 를 SCLK 상승 edge 마다 샘플해 바이트를 재구성, CS_N 상승 시 `miso_last_byte`/`miso_byte_ready` 갱신. TEST 2/3/5/6/7 의 TX 판정 근거.
- TX_DEBUG 모니터(L217–238): 4,800–15,000 ns 구간에서 40 ns 간격(SCLK 상승 edge)으로 CS, STATE, BIT, HOLD, VALID, CUR, SHIFT, POP, MISO 를 출력 — LOG-13 에서 255 샘플 확인, 그림 7-2/7-3 재구성의 원자료.
- TEST 8/9 는 TB 안에 별도 `async_fifo` 인스턴스(`test8_fifo`)를 두고 Gray 포인터 전이 checker(L300–348)와 push/pop task(L350–395)로 구동.

5.4 Testbench 적절성 평가 (요약)

관점	평가	근거
Self-checking	양호 — 모든 TEST 가 자동 판정, 마지막에 오류 개수 요약	check_eq + errors 카운터 확인

관점	평가	근거
독립 관측	양호 — MISO 를 DUT 내부 신호가 아니라 출력 핀(`o_dbg_miso`)에서 재구성	L158–209 확인
내부 신호 의존	주의 — `dut.u_register_block.r_drain_active`, `dut.w_rx_fifo_full`, `dut.w_tx_fifo_wr_en`, `dut.u_spi_master.state` 등 계층 참조 다수. RTL 이름 변경에 취약, 외부 인터페이스 검증이 아님	TEST 4/5/6/10 확인
데이터 비교	부분적 — RX 순서(TEST 1), TX 값(2/3/5/6/7), FIFO 128 byte(9)는 비교. **TEST 4 는 값을 비교하지 않음**	TB 코드 분석
Scoreboard	없음 — queue 기반 기대값 모델은 중간 TB(093905)에서만 사용	LOG-093905
랜덤/제약 랜덤	없음 — 모든 stimulus 가 directed. TEST 9 의 데이터는 결정적 수식	TB 코드
Assertion / 프로토콜 checker	없음 — SPI Mode 0 의 setup/hold, CS_N 유효 구간 등을 별도로 검사하지 않음	TB 코드
시나리오 다양성	한정 — master 가 내장이라 외부 master 의 지연·back-to-back·mode 변형 없음. SCLK 는 항상 ACLK÷4	RTL 구조
정량 coverage	미측정	8장

> **종합.** 최종 TB 는 “설계 의도대로 흐르는 경로가 동작하는가”를 directed 방식으로 잘 확인하지만, 결함을 **적극적으로 찾는** 구조(랜덤, scoreboard, assertion)는 아니다. RTL-008 이 최종 TB 를 통과한 것이 그 증거다.

6 TEST 1 ~ TEST 10 상세

각 TEST 를 10개 항목으로 정리합니다. “로그 증거”는 최종 run(LOG-13 L2929–3287)에서 인용했습니다. 값은 로그의 16진 출력을 그대로 옮겼습니다(예: `TEST4 RX FIFO LEVEL = 10` 은 16, `TEST8 write count = 20` 은 32).

6.1 TEST 1 — RX path end-to-end

1. 목적	master 가 보내는 SPI 프레임이 slave → RX FIFO → prefetch → `RX_DATA` 레지스터 경로로 순서대로 도착하는지 확인.
2. 시나리오	reset 해제 후 `STATUS[3]` (RX_VALID)를 폴링하다가 1 이면 `RX_DATA` 를 읽고 기대값과 비교. 6 프레임 반복.
3. Stimulus	master 내장 TX 패턴 0,1,2,…(자동 증가). 버스 read 는 `bus_read` task.
4. 기대값	읽은 바이트가 0,1,2,3,4,5 순서.
5. Checker	`check_eq("RX_DATA_REG sequential byte", expected_rx_byte, rx_val)`
6. 로그 증거 (LOG-13 마지막 run)	`[195000] TEST 1 → [1115000] … = 0, [1815000] =1, [2735000] =2, [3435000] =3, [4135000] =4, [4835000] =5`
7. 결과	**PASS ×6** 확인 (195 → 4,835 ns)
8. TB 적절성 평가	RX 경로의 기본 정합성은 확인. 폴링 후 즉시 읽으므로 FIFO backlog 가 거의 없다.
9. 한계	① backlog ≥ 2 상태 미검증 → RTL-008 검출 불가. ② STATUS 의 나머지 비트(FULL/BUSY)는 값 확인 안 함. ③ 기대값 0–5 는 하위 3비트만 사용하므로 **상위 비트(bit 7~3)의 고착·오류를 검출하지 못함** (TEST 4 의 22 프레임에서 값이 커지지만 비교하지 않음).
10. 관련 이슈	RTL-003 (FIFO), RTL-008 (미해결)

6.2 TEST 2 — TX path end-to-end

1. 목적	버스에서 쓴 바이트가 TX FIFO → hold → MISO 로 나가는지 확인.
2. 시나리오	`TX_DATA` 에 0x5A 쓰기 → 최대 4개의 프레임에서 CS_N 상승마다 MISO 재구성 값을 확인.
3. Stimulus	`bus_write(ADDR_TX_DATA, 8'h5A)`
4. 기대값	4프레임 안에 `miso_last_byte === 8'h5A`.
5. Checker	`check_eq("TX_DATA_REG to MISO", 1, tx_match_found)`
6. 로그 증거 (LOG-13 마지막 run)	`[4835000] TEST 2 → [5486000] TX frame 1: MISO = 0x5a READY=1 → PASS: TX_DATA_REG to MISO = 1`
7. 결과	**PASS ×1** 확인. 같은 TEST 가 RUN1–4 에서는 `[8526000] FAIL: TX_DATA_REG to MISO expected=1 got =0`.
8. TB 적절성 평가	MISO 를 출력 핀에서 독립 재구성하므로 DUT 내부 가정에 덜 의존한다.
9. 한계	**0x5A(0101 1010)는 비트열이 대칭이라 MSB/LSB 순서 오류를 검출하지 못한다.** 비트 순서는 TEST 3 의 0x11/0x22/0x33 이 담당. 4 프레임 허용이라 “몇 번째 프레임에 나오는가”는 정확히 검사하지 않음.
10. 관련 이슈	TB-003, RTL-006

6.3 TEST 3 — TX burst (0x11 → 0x22 → 0x33)

1. 목적	연속 쓰기한 3바이트가 순서대로, 중복·손실 없이 3개의 프레임으로 나가는지 확인 (TX FIFO + hold 의 핵심 검증).
2. 시나리오	TX_DATA 에 0x11, 0x22, 0x33 을 연속 write 한 뒤 CS_N 상승마다(최대 8프레임) MISO 확인.
3. Stimulus	`bus_write(ADDR_TX_DATA, 8'h11/8'h22/8'h33)`
4. 기대값	프레임 순서대로 0x11, 0x22, 0x33 (첫 바이트 도착 전의 0x00 은 INFO 로 허용).
5. Checker	프레임별 비교 후 `PASS: TX Burst byte n`, 최종 `PASS: TX Burst sequence 11 -> 22 -> 33`

6. 로그 증거 (LOG-13 마지막 run)	`[6246000] PASS … byte 1 = 0x11, [7006000] byte 2 = 0x22, [7766000] byte 3 = 0x33, [7766000] PASS: TX Burst sequence 11 -> 22 -> 33`
7. 결과	**PASS ×4** 확인. 프레임 간격 760 ns.
8. TB 적절성 평가	이전 RTL 의 0x33×16 결함(RTL-006)을 정확히 겨냥한 회귀 TEST. 비대칭 값(0x11, 0x22, 0x33)이라 비트 순서도 확인.
9. 한계	3 byte 만, 프레임 중간에 쓰기가 들어오는 경우·FIFO 가 비는 경우 없음. 0x00 허용 규칙이 첫 바이트의 지연 도착을 가림.
10. 관련 이슈	RTL-006 (해결)

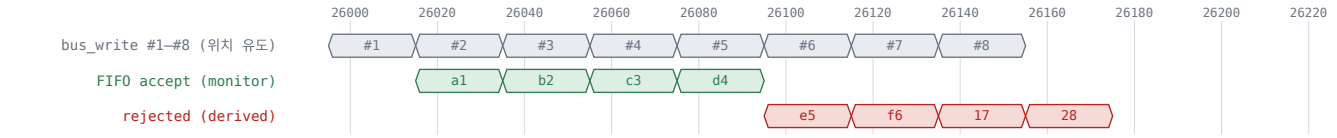
6.4 TEST 4 — RX FIFO full / overflow

1. 목적	RX FIFO 가 가득 찼을 때 FULL, level, OVERFLOW sticky, 출력 포트가 올바르고 clear 후 풀리는지 확인.
2. 시나리오	`CONTROL=0x03` 으로 drain+overflow clear → 읽지 않고 22 프레임 대기 → STATUS/LEVEL/overflow 확인 → `CONTROL=0x01` (FIFO_CLEAR) → FULL 해제 확인 → `CONTROL=0x02` (OVF_CLEAR) → OVERFLOW 해제 확인.
3. Stimulus	버스 write/read, 계층 대기 `dut.u_register_block.r_drain_active`.
4. 기대값	FULL=1, level=16, STATUS[4]=1, `o_overflow`=1 → drain 후 FULL=0, OVERFLOW=0.
5. Checker	`check_eq` ×6 (FULL 은 `dut.w_rx_fifo_full` 직접 probe, level/STATUS 는 버스 read)
6. 로그 증거 (LOG-13 마지막 run)	`[25405000] TEST4 status=1a level=16 overflow=1` (0x1A = OVERFLOW·RX_VALID·FULL 비트), `PASS: TEST4 RX FIFO FULL = 1, LEVEL = 10 (16), OVERFLOW sticky = 1, OVERFLOW output = 1, [25665000] FULL cleared after drain = 0, [25745000] OVERFLOW cleared = 0`
7. 결과	**PASS ×6** 확인 (7,766 → 25,745 ns)
8. TB 적절성 평가	overflow 가 SCLK→ACLK pulse_sync 를 거쳐 sticky 로 잡히는 경로를 실제로 통과시킨다. 22 프레임 > 용량 (16+prefetch 1)으로 overflow 보장.
9. 한계	① **값(어떤 바이트가 남고 버려졌는지) 비교 없음.** ② FULL 은 STATUS[1] 이 아니라 내부 probe 로 확인 → CDC-01 경로 (STATUS[1])는 직접 검증되지 않음. ③ drain 대기가 계층 참조.
10. 관련 이슈	RTL-008 (검출 못함), CDC-01

6.5 TEST 5 — TX FIFO full / boundary

1. 목적	TX FIFO(4 entry)가 가득 찼을 때 추가 쓰기가 거절되고, 수락된 바이트는 순서대로 전송되는지 확인.
2. 시나리오	CS_N 상승 후 8바이트(A1,B2,C3,D4,E5,F6,17,28)를 연속 write, `dut.w_tx_fifo_wr_en` 로 수락 수를 센 뒤 이후 8 프레임의 MISO 를 수락 순서와 비교.
3. Stimulus	`bus_write ×8` (1 cycle 간격)
4. 기대값	수락 ≥1, 수락 < 시도(=full 발생), 수락 바이트가 순서대로 전송.
5. Checker	카운터 기반 조건 3개 + 순서 비교 → `PASS: TEST5 FIFO Full observed`, `order[i]`
6. 로그 증거 (LOG-13 마지막 run)	`[26025000] ACCEPT a1, [26045000] b2, [26065000] c3, [26085000] d4, TEST5 ATTEMPTED=8 ACCEPTED=4 NOT_ACCEPTED=4 FULL_CYCLES=11, PASS order[0..3]=a1,b2,c3,d4`
7. 결과	**PASS ×5** 확인 (25,745 → 32,087 ns). 그림 6-1.
8. TB 적절성 평가	“full 이 실제로 관측되었는가(FULL_CYCLES=11)”와 “수락된 것만 순서대로 나가는가”를 모두 본다 — 공허한 PASS 를 피하는 좋은 설계.
9. 한계	① 수락 판정이 `dut.w_tx_fifo_wr_en` probe(registered 신호) 기반. ② 거절된 write 는 **SW 가 알 수 없다** — STATUS 에 TX FULL 비트가 없음(설계 관찰 확인). ③ 수락 개수 4 는 FIFO 깊이 4 와 일치하지만 hold register 에 1 byte 가 이미 빠져 있었을 가능성은 이 로그로 구분 불가 미확인.
10. 관련 이슈	—

그림 6-1 TEST 5 — TX FIFO(4-deep) full 경계: 8회 write 중 4회 수락 (LOG-13 + TB 코드)
초록 = 로그 [26025000]…[26085000] TEST5 FIFO ACCEPT a1/b2/c3/d4 (확인). 적색/회색 = TB 코드의 20 ns 간격 연속 write 와 로그의 ATTEMPTED=8 ACCEPTED=4 로부터 유도(위치는 유도값).



확인(로그): ATTEMPTED=8 ACCEPTED=4 NOT_ACCEPTED=4 FULL_CYCLES=11, PASS: FIFO Full observed, order[0..3]=a1,b2,c3,d4.
유도(추정): 거절된 write 는 마지막 4개(E5,F6,17,28). 로그에는 거절 바이트 값이 직접 찍히지 않으며 write 정확 시각도 찍히지 않음.

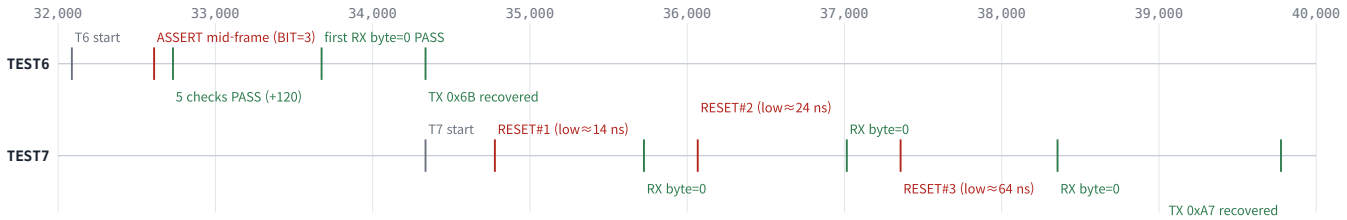
6.6 TEST 6 — Reset during transfer

1. 목적	프레임 전송 도중 ARESETn 이 걸렸을 때 모든 상태가 초기화되고 이후 정상 동작이 복귀하는지 확인.
2. 시나리오	TX 에 0x96 write → master 가 SHIFT, `bit_cnt≥3` 가 되면 ARESETn=0 → 12 negedge 후 검사 → 해제 → 첫 RX 바이트와 TX 0x6B 확인.
3. Stimulus	ARESETn, `dut.u_spi_master.state/bit_cnt` 대기
4. 기대값	리셋 중 CS_N=1, RX/TX FIFO empty, overflow 0, RX_VALID 0. 복귀 후 첫 RX 바이트=0, TX 0x6B 정상.
5. Checker	`check_eq` ×7
6. 로그 증거 (LOG-13 마지막 run)	`[32610000] TEST6 ASSERT RESET mid-frame CS=0 BIT=3` → `[32730000] PASS` ×5 → `[33675000] first RX byte after reset = 0` → `[34336000] TEST6 recovery MISO frame 1 = 6b, TX recovered after reset = 1`. RUN9 에서는 `[32730000] FAIL: TEST6 CS inactive in reset expected=1 got=0`.
7. 결과	**PASS ×7** 확인 (32,087 → 34,336 ns). 그림 6-2.
8. TB 적절성 평가	RTL-007 을 정확히 노출시킨 TEST. 복귀 후 “master 패턴이 0 부터 재시작” 까지 확인.
9. 한계	reset 지점이 `bit_cnt≥3` 한 곳. TX 가 이미 FIFO 에 있을 때의 reset(0x96) 은 “FIFO empty” 만 확인하고 hold 레지스터 값은 직접 확인하지 않음. 계층 참조 사용.
10. 관련 이슈	RTL-007 (해결)

6.7 TEST 7 — Repeated reset / release timing

1. 목적	짧은 reset 펄스(≈14, 24, 64 ns)를 프레임 도중 3회 반복 인가해도 복구되는지 확인.
2. 시나리오	프레임 시작 후 3 SCLK 뒤 ARESETn=0 → 1 ns 후 CS_N·overflow 확인 → 지정 시간 후 FIFO empty 확인 → 다음 ACLK negedge 에서 해제 → 첫 RX 바이트(0) 확인. 3회 후 TX 0xA7 복구.
3. Stimulus	펄스 길이 #6/#12/#54 (TB 주석 7/13/55 ns, 실제 low ≈14/24/64 ns 추정)
4. 기대값	CS_N=1(+1 ns), overflow 0, FIFO empty, 55 ns 펄스에서 RX_VALID=0, 복귀 후 첫 RX=0, TX 0xA7.
5. Checker	`check_eq` ×17
6. 로그 증거 (LOG-13 마지막 run)	`RESET #1/#2/#3 asserted` @34,777 / 36,067 / 37,357 ns, FIFO reset 검사 @34,784 / 36,080 / 37,412 ns, 첫 RX byte @35,725 / 37,015 / 38,355 ns, `[39776000] TEST7 recovery TX frame 1 = a7`. RUN11 에서는 `RX valid reset expected=0 got=1` ×3.
7. 결과	**PASS ×17** 확인 (34,336 → 39,776 ns).
8. TB 적절성 평가	여러 폭의 reset 에서 복구되는 것을 확인. 그림 6-2.
9. 한계	**RX_VALID 즉시-0 검사는 55 ns 펄스에만 적용**(TB-004). 짧은 reset 에서 register_block(sync reset)의 즉시 응답은 검사하지 않음. 최단 reset 폭이 1 ACLK 미만인 경우, 비정수 위상 reset 등은 없음. 라벨 앞글자 잘림(`EST7`).
10. 관련 이슈	RTL-007, TB-004, TB-005

그림 6-2 TEST 6 / TEST 7 — reset 이벤트 시간축 (LOG-13에서 읽은 시각, ns)



확인: 각 reset 직후 +1 ns 에 CS_N=1, overflow=0 PASS (34778, 36068, 37358 ns). FIFO empty 확인은 +7 / +13 / +55 ns (34784 / 36080 / 37412).
유도(추정): reset low 폭 = TB가 "다음 ACLK negedge +1 ns" 에서 해제하므로 실제 14 / 24 / 64 ns (주석의 7 / 13 / 55 ns 는 해제 전 대기 구간). ACLK negedge 가 10 ns 배수라는 가정.
[한계] 이 로그는 이벤트/체크 시각만 담고 있음. ARESETn, aclk_rst_n, sclk_rst_n 의 실제 파형은 증거에 없음 → 파형을 그리지 않음(미확인).
[한계] 7 / 13 ns 급 짧은 펄스에서 ACLK 도메인 reset 가시성은 보장되지 않는다는 점을 TB 코드 주석이 이미 인정(TEST7 RX valid 체크를 55 ns 펄스에만 적용).

6.8 TEST 8 — Async FIFO CDC / Gray pointer

1. 목적	`async_fifo` 모듈을 서로 다른 클럭(10 ns / 14 ns)으로 구동해 full·empty·wrap 과 Gray 포인터 단일 비트 전이를 확인.
2. 시나리오	TB 내부 `test8_fifo` 에 16 write → full 확인 → 추가 write 차단 확인 → 16 read(0x40+i) → empty/level 0 → 16 write(0x80+i) 후 16 read (wrap) → Gray 오류 0, write/read 카운트 32.
3. Stimulus	`test8_push/test8_pop` task, wr_clk=ACLK, rd_clk=`test8_rd_clk`(14 ns)
4. 기대값	initial empty=1/full=0, full at 16, 추가 write 후에도 `r_wr_ptr_bin`=16, empty after reads, level 0, Gray 오류 0, 카운트 32/32.
5. Checker	`check_eq` ×10 + Gray checker(`test8_gray_errors`)
6. 로그 증거 (LOG-13 마지막 run)	`[39870000] initial empty=1, [40191000] full at 16 writes = 1, [40230000] full blocks extra write = 10`(16), `[40740000] empty after 16 reads, [41566000] Gray single-bit transitions = 0, write count = 20`(32)
7. 결과	**PASS ×10** 확인 (39,776 → 41,566 ns). 그림 6-3.
8. TB 적절성 평가	비정수비 클럭에서 포인터 전달과 wrap 을 보는 점이 좋다. Gray 전이 checker 는 CDC 설계 의도를 직접 검사한다.
9. 한계	① **DUT 내부 FIFO 가 아닌 별도 인스턴스** — top 안의 RX/TX FIFO(ADDR_WIDTH 4/2, SCLK/ACLK 관계)를 직접 검증한 것이 아님. ② 모의 환경이 metastability 를 모델링하지 않음. ③ 계층 참조(`test8_fifo.r_wr_ptr_bin`).
10. 관련 이슈	RTL-001/002/003

그림 6-3 5-bit 포인터(ADDR_WIDTH=4)의 Binary → Gray 변환과 full 조건 (RTL 수식에서 계산한 값)

wr_ptr_bin	00	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17
wr_ptr_gray	00000	00001	00011	00010	00110	00111	00101	00100	01100	01101	01111	01110	01010	01011	01001	01000	11000	11001
Δ(Gray) 비트		b0	b1	b0	b2	b0	b1	b0	b3	b0	b1	b0	b2	b0	b1	b0	b4	b0

`full 조건 (RTL line 874): w_wr_ptr_gray_next == { ~rd_gray_wclk[4:3], rd_gray_wclk[2:0] }`
예) rd=0 (00000), wr 가 16 에 도달 → wr_gray=11000 = {~00, 000} → full=1 (TEST8: 16회 write 후 full, 로그 [40191000]).
`empty 조건 (RTL line 912): w_rd_ptr_gray_next == r_wr_ptr_gray_rclk (reset 값 empty=1)`

확인: 인접 값의 Gray 코드는 항상 1 비트만 다르다(위 Δ 행 = 계산 결과). TEST8 은 DUT 가 아니라 test8_fifo 내부 레지스터 r_wr_ptr_gray / r_rd_ptr_gray 의 변화량을 매 클럭 검사.
[한계] 검사 대상은 각 클럭 도메인 안의 포인터 전이. 2-FF 로 상대 도메인에 샘플링되는 순간의 skew/메타스터빌리티는 시뮬레이션 모델에 없음 → 정적 CDC 필요.
계산: 31→0 wrap 도 gray(31)=10000 → gray(0)=00000 로 b4 한 비트만 변화(1비트 전이 성질은 wrap 에서도 유지).

6.9 TEST 9 — Concurrent FIFO read / write stress

1. 목적	독립 클럭에서 쓰기·읽기를 동시에 불규칙하게 수행해도 데이터가 순서·값 모두 보존되는지 확인.
2. 시나리오	`test8_fifo` 에 128 byte: writer 는 `written%7==3` 에서 한 cycle 쉬고, reader 는 `read%5==2` 에서 쉼. 읽을 때마다 기대값 수식으로 비교.
3. Stimulus	데이터 `((n*37) ^ (n>>2) ^ 8'hA5) & 8'hFF`
4. 기대값	128 write, 128 read, 모든 읽기값 일치, 종료 후 empty/level 0.
5. Checker	`test8_rd_data !== test9_expected` 시 `errors++`; `check_eq` ×5

6. 로그 증거 (LOG-13 마지막 run)	`[41692000] initial empty = 1, [43988000] writes = 80(128), reads = 80(128), FIFO empty = 1, FIFO level = 0`
7. 결과	**PASS ×5** 확인 (41,566 → 43,988 ns)
8. TB 적절성 평가	불규칙 간격으로 full/empty 경계를 반복 통과시켜 동시 접근의 정합성을 폭넓게 확인.
9. 한계	데이터 비교 결과는 **PASS 줄로 출력되지 않고 errors 에만 반영** — 성공 시 로그에서 비교가 실제로 수행되었는지 확인할 수 없음. 패턴이 결정적(랜덤 아님). 별도 인스턴스.
10. 관련 이슈	—

6.10 TEST 10 — Register / protocol corner cases

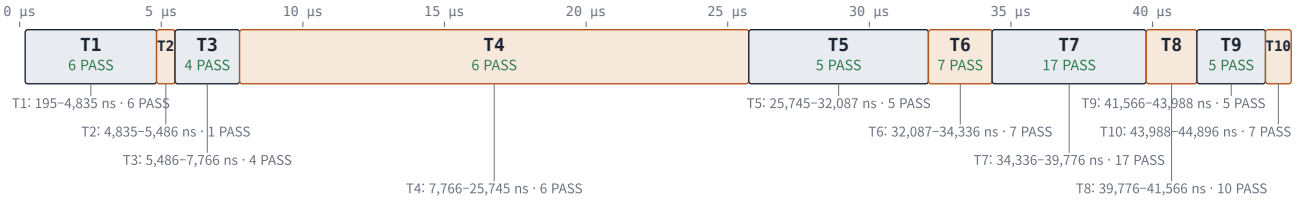
1. 목적	레지스터 corner case(미사용 주소, CONTROL readback, drain 종료, overflow clear)와 reset 후 SPI 프레임 완료.
2. 시나리오	ARESETn 한 번 인가 → 주소 0xFE 읽기 → CONTROL=0x04/0x00 readback → FIFO_CLEAR 후 drain 종료 → OVF_CLEAR → 다음 프레임 완료.
3. Stimulus	버스 read/write, ARESETn(8 negedge)
4. 기대값	0xFE 읽기=0, readback 4 → 0, drain 종료, overflow 0, CS_N idle, `miso_byte_ready`=1.
5. Checker	`check_eq` ×7
6. 로그 증거 (LOG-13 마지막 run)	`[44175000] unmapped address reads zero = 0, [44205000] IRQ enable readback = 4, [44235000] IRQ disable readback = 0, [44260000] drain finishes = 0, [44320000] overflow clear = 0, [44896000] CS idle after frame = 1, completed SPI frame = 1`
7. 결과	**PASS ×7** 확인 (43,988 → 44,896 ns)
8. TB 적절성 평가	레지스터 map 의 default 분기(미사용 주소)와 CONTROL readback 을 실제로 확인. IRQ_ENABLE 은 저장/readback 만 있음을 TEST 가 그대로 반영.
9. 한계	IRQ 출력이 없어 “IRQ 동작” 은 검증 불가. `FIFO_LEVEL`, TX_DATA readback 값은 검사하지 않음. 쓰기 중 읽기 등 동시 버스 접근 없음.
10. 관련 이슈	—

7 시뮬레이션 결과와 증거

7.1 최종 실행 개요

실행	LOG-13 의 13번째(마지막) `Running xsim` — L2929–3287
결과	`[44896000] ALL TESTS PASSED (tb_spi_bridge_top), $finish called at time : 44896 ns … Line 1066`
check 줄	PASS 68 / FAIL 0 (개별 check 줄 기준. “ALL TESTS PASSED” 줄은 제외) 확인
시각 단위	로그 `[%0t]` = ps. 값 출력은 16진(예: LEVEL = 10 → 16)

그림 7-1 최종 XSim 실행(RUN13) 시간 구간별 TEST 진행 — LOG-13 마지막 run, 0–44,896 ns



T1 = RX end-to-end
T2 = TX end-to-end
T3 = TX burst
T4 = RX FIFO full/overflow
T5 = TX FIFO full/boundary
T6 = mid-frame reset
T7 = repeated reset
T8 = Async FIFO Gray
T9 = concurrent stress
T10 = register/protocol

[확인] 총 68 PASS 줄 / FAIL 0 / 종료 44,896 ns ($finish, tb_spi_bridge_top.v Line 1066). TEST5 PASS 줄은 로그에 타임스탬프가 없음.

TEST	시작 (ns)	종료 (ns)	PASS	핵심 로그
1 RX end-to-end	195	4,835	6	RX_DATA 0..5
2 TX end-to-end	4,835	5,486	1	MISO=0x5a
3 TX burst	5,486	7,766	4	11→22→33
4 RX FIFO full/overflow	7,766	25,745	6	status=1a level=16 overflow=1
5 TX FIFO boundary	25,745	32,087	5	ACCEPTED=4 / FULL_CYCLES=11
6 Reset during transfer	32,087	34,336	7	RESET @32,610, recovery 0x6b
7 Repeated reset	34,336	39,776	17	RESET @34,777/36,067/37,357, recovery 0xa7
8 Async FIFO	39,776	41,566	10	full@16, Gray errors 0, 32/32
9 Concurrent FIFO	41,566	43,988	5	128 writes / 128 reads
10 Corner cases	43,988	44,896	7	unmapped=0, IRQ readback 4/0

7.2 RUN 이력 (LOG-13 안의 13 회 실행)

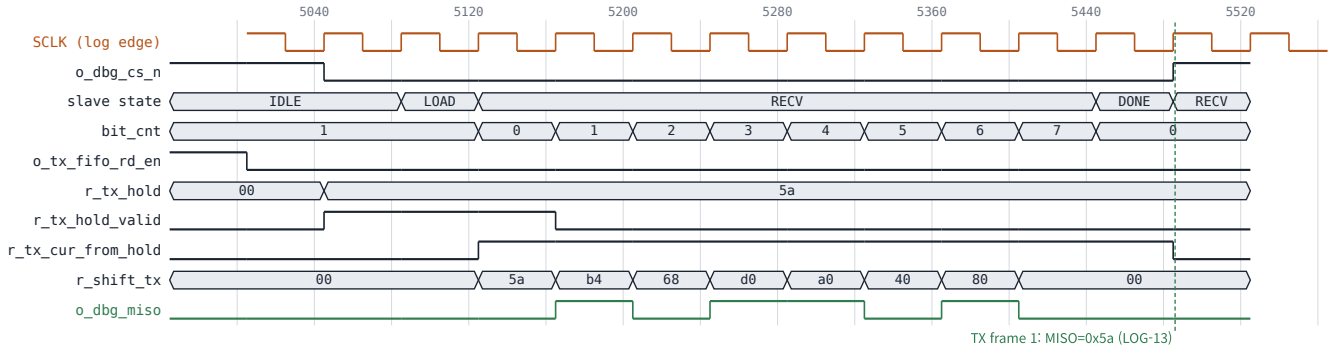
RUN	결과	핵심 줄	해석
1–4	FAIL	`[8526000] FAIL: TX_DATA_REG to MISO expected=1 got=0`, `TX frame 1: MISO = 0x00` (6,246 ns)	TX 경로 미완 상태 4회 반복 확인
5	PASS	`[5486000] TX frame 1: MISO = 0x5a → ALL TESTS PASSED`	TX hold 도입 후 추정 (RTL 리비전 직접 증거 없음)
6	PASS	`[7766000] ALL TESTS PASSED`	TEST 3 (burst) 추가
7	PASS	`[25745000] ALL TESTS PASSED`	TEST 4/5 추가
8	PASS	`[32095000]`, 라벨 “TEST 1: RX Sequential”	디버그 출력 포함 별도 TB 버전. 최종 TB 와 동일하지 않음
9	FAIL	`[32730000] FAIL: TEST6 CS inactive in reset expected=1 got=0`	RTL-007
10	PASS	`[34336000] ALL TESTS PASSED`	async reset 적용 후

RUN	결과	핵심 줄	해석
11	FAIL ×3	`TEST7 RX valid reset expected=0 got=1` @34,778 / 36,068 / 37,358	TB-004
12	PASS	`[39776000] ALL TESTS PASSED`	TEST 7 검사 수정 후
13	PASS	`[44896000] ALL TESTS PASSED`	TEST 8–10 추가, 최종

7.3 재구성 파형 (스크린샷이 아닌 작성자 재구성)

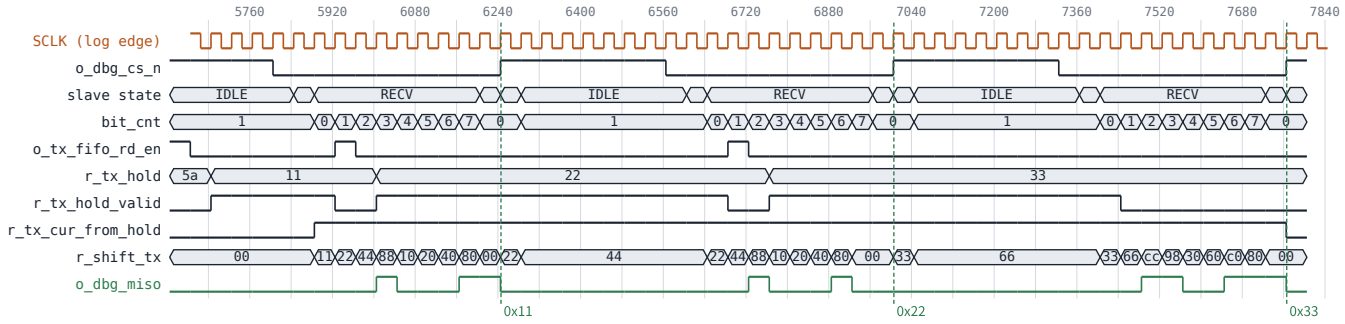
방법과 한계 재구성. 최종 run 의 `TX_DEBUG` 출력은 SCLK 상승 edge(40 ns 간격)마다 CS, STATE, BIT, HOLD, VALID, CUR, SHIFT, POP, MISO 를 한 줄씩 찍는다. 각 샘플 k 는 직전 구간 [t_k−40, t_k) 의 값(NBA 이전 값)을 나타낸다고 해석했고, 이 해석은 Vivado 스크린샷(그림 4-9: hold_valid 5,045–5,165 ns, state=1 5,085–5,125 ns)과 1 ns/px 수준으로 일치함을 확인했다 확인. 아래 파형은 그 40 ns 해상도 샘플로 그린 것이므로 **40 ns 보다 짧은 glitch·skew 는 표현되지 않으며**, 시뮬레이터가 출력한 파형 자체가 아니다.

그림 7-2 TEST 2 — TX frame 1 (0x5A) 재구성 파형: 로그 TX_DEBUG 샘플 기반
각 칸 = 해당 SCLK 상승 에지 직전 1주기 값(40 ns 해상도). 전이 시각은 칸 경계(±40 ns). 시뮬레이터 캡처가 아니라 LOG-13 줄에서 재구성.



확인: MISO=1 인 칸의 bit_cnt = 1, 3, 4, 6 → 0x5A = 0101_1010 의 값 1 인 비트(b6, b4, b3, b1)와 일치. BIT0 칸 MISO=0(b7), 마지막 칸 MISO=0(b0).
확인: hold 적재(POP→HOLD=5a, 5085 ns), CS_N 하강(5085), LOAD(5125), RECEIVE 8 bit(5165–5445), BYTE_DONE(5485). 같은 구간의 Vivado 파형(그림 4-9/4-10)과 일치.

그림 7-3 TEST 3 — TX burst 0x11 → 0x22 → 0x33 재구성 파형 (LOG-13 TX_DEBUG)
r_tx_hold 가 프레임 중 다음 바이트(0x22, 0x33)로 미리 채워지고, consume 이후(BIT=1) 다시 pop 되는 prefetch 동작이 보인다.



확인: 프레임당 SCLK 19주기(CS_N 하강 간격 5845→6605→7365 ns = 760 ns). 확인: HOLD 교체는 BIT=3(6045 ns, 6805 ns)에서 일어나 현재 바이트 shift 를 건드리지 않음.
해석(추정): POP→HOLD 갱신이 BIT=1→3 사이에 일어나는 것은 r_tx_pop_d 1주기 지연 + async FIFO 읽기 레지스터 지연의 합. 정확한 사이클 귀속은 파형 확대로 추가 확인 필요.

7.4 파형 증거의 사용 원칙

구분	사용한 것	사용하지 않은 것과 이유
실제 스크린샷(파형 영역만 crop)	그림 4-2 ~ 4-11 (SimVision 2, Vivado 8)	31장 중 나머지: 신호가 30개 이상 한 화면에 압축되어 확대 없이는 판독 불가(예: 101830/101833, 062848)이거나 콘솔·대화상자(095716 “Invalid Top Module”). “파형만 보이게” crop 하되 **신호 이름·값 칸은 남김**(어떤 신호인지 알아야 하므로)
재구성 파형 재구성	그림 6-1, 6-2, 7-2, 7-3, 6-3(Gray 표)	로그 값·RTL 수식 기반. 시뮬레이터 출력 아님
확인 불가	TEST 8/9 의 내부 포인터 파형, overflow pulse_sync 파형, reset 구간 SCLK/CS_N 의 최종-run 파형	해당 구간의 읽을 수 있는 캡처 없음 미확인

8 Coverage, 한계, Verification Matrix

> **정량 Coverage 미측정.** 코드(line/branch/toggle/FSM) coverage, functional coverage, assertion coverage 어느 것도 측정하지 않았습니다. 따라서 68 PASS 는 “68개 check 가 통과”라는 의미이며 coverage 퍼센트로 해석해서는 안 됩니다. 아래 Matrix 는 **정성적** 정리입니다.

8.1 Verification Matrix

검증 항목	TEST	결과	상태	비고
RX 바이트 순서 (SPI→RX_DATA)	1	PASS ×6	부분	값 0–5, backlog 없음
RX_DATA 읽기 시 pop / valid 해제	1	PASS(간접)	부분	backlog ≥2 시 손실(RTL-008)
TX 단일 바이트 → MISO	2	PASS ×1	부분	0x5A 대칭값
TX 3 byte burst 순서	3	PASS ×4	검증됨	3 byte 한정
MISO 비트 순서(MSB-first)	3, 5	PASS	검증됨	0x11/0x22/0x33, A1..D4 비대칭값
RX FIFO full / level=16	4	PASS ×2	검증됨	FULL 은 내부 probe
OVERFLOW sticky / 출력 / clear	4, 6, 7	PASS	검증됨	동일
FIFO_CLEAR drain (RX)	4, 10	PASS	부분	TX FIFO 는 clear 대상 아님(설계)
RX 데이터 값 보존(overflow 시)	4	—	미검증	값 비교 없음
TX FIFO full 경계 / 순서	5	PASS ×5	검증됨	수락 4/8
TX full 을 SW 가 알 수 있는가	—	—	미검증	STATUS 에 비트 없음(설계 한계)
프레임 중 reset 복구	6	PASS ×7	검증됨	1개 지점
반복·짧은 reset 복구	7	PASS ×17	부분	RX_VALID 즉시 응답은 55 ns 만
async_fifo full/empty/wrap	8	PASS ×10	부분	별도 인스턴스
Gray 포인터 단일 비트 전이	8	PASS	부분	시뮬레이션 checker, 별도 인스턴스
동시 읽기/쓰기 데이터 보존(128 B)	9	PASS ×5	부분	directed 패턴, 별도 인스턴스
미사용 주소 읽기=0	10	PASS	검증됨	0xFE 한 주소
CONTROL.IRQ_ENABLE readback	10	PASS ×2	부분	IRQ 출력 없음
STATUS.SPI_BUSY / EMPTY 비트 값	—	—	미검증	값 비교 없음
SPI Mode 0 타이밍 (setup/hold, CS_N 구간)	—	TX_DEBUG 로 간접	미검증	protocol checker 없음
외부 SPI master 호환 / 임의 SCLK	—	—	미검증	내장 master 만
rx_full(CDC-01) 경로	—	—	미검증	STATUS[1] 읽기는 미검사
backlog ≥2 RX 연속 읽기	—	—	미검증	RTL-008 이 이 영역
랜덤 / 장시간 regression	—	—	미검증	
Lint	—	—	미수행	
정적 CDC / RDC	—	—	미수행	
SDC / 합성 (최종 RTL)	—	—	미수행	다른 리비전 합성만 존재
STA (최종 RTL)	—	—	미수행	
LEC	—	—	미수행	
P&R / PD	—	—	미수행	
정량 Coverage	—	—	미수행	

8.2 한계 요약

- **RTL-008 미해결**이며 최종 TB 가 이를 검출하지 못한다.
- **CDC-01/02** 열림 — 기능 PASS 가 CDC 안전성을 보장하지 않는다.
- TEST 8/9 는 **top 내부 FIFO 가 아닌 별도 인스턴스**를 검증한다.
- 계층 참조 의존, directed only, assertion 없음.
- SCLK 는 ACLK 에서 생성되는 free-running 클럭 — 외부 SPI master 환경 미검증.
- 합성 증거는 다른 RTL 리비전이며 CDC set_max_delay 는 비활성. 각 RUN 이 사용한 정확한 RTL 리비전도 증거로 보장되지 않음.

9 Lessons Learned

#	교훈	근거
1	**flag 는 register 로, 루프는 구조로 끊는다**	FIFO full/empty 를 다음 포인터에서 조합 생성하던 구조(RTL-001)는 flag 를 register 로 바꾸며 해결했다. 조합 루프는 시뮬레이션에서 X 고착, 합성에서 loop-breaking 필요성으로 나타나므로 "돌아간다"가 합격 기준이 아니다.
2	**"PASS" 가 공허하지 않은지 먼저 본다**	결함 있는 FIFO 에서도 "drain 후 empty 확인" 이 PASS 했다(TB-002). 최종 TB 의 TEST 5 는 FULL_CYCLES 를 세어 "full 이 실제로 일어났는가"를 확인하는 방식으로 이를 보완했다.
3	**리셋을 만드는 클럭이 리셋 때문에 멈추면 async assert**	SCLK 가 reset 중 정지하는 구조에서 sync reset 은 적용되지 않는다(RTL-007). 클럭 생성·분주 구조를 먼저 그리고, 각 도메인의 reset 이 어떤 클럭으로 동작하는지 표로 확인해야 한다.
4	**FIFO 읽기는 "요청 중" 상태를 가진다**	TX 쪽은 `r_tx_pop_d` /hold 로 중복 pop 을 막았지만 RX 쪽 대칭 보호가 빠져 RTL-008 이 남았다. 같은 패턴은 모든 registered-read FIFO 소비자에 필요하다.
5	**테스트 데이터는 비대칭, 비자명하게**	0x5A 는 비트 순서를 뒤집어도 같아 TEST 2 만으로는 MSB/LSB 오류를 못 본다. 0–5 는 상위 비트가 항상 0 이다. 데이터 선택 자체가 검증 품질이다.
6	**TB 의 주석이 아니라 코드를 믿는다**	TEST 7 주석은 7/13/55 ns 라고 적었지만 실제 low 폭은 약 14/24/64 ns 다(TB-005). 로그 시각으로 교차 확인했다.
7	**계층 참조는 편하지만 인터페이스 검증이 아니다**	`dut.*` 참조가 많을수록 TB 가 RTL 구현에 묶인다. 외부 포트로 관찰 가능한 것은 포트로 검증한다.
8	**기능 시뮬레이션과 CDC 검증은 다른 활동이다**	Gray 단일 비트 전이는 시뮬레이션으로 확인할 수 있으나 metastability/reconvergence 는 정적 도구의 영역이다. 보고서에서 두 가지를 섞어 쓰지 않았다.
9	**증거는 버전과 함께 보관한다**	어떤 로그가 어떤 RTL 에서 나왔는지 증명할 수 없어 여러 곳에서 "추정"으로 남겼다. RTL·TB·로그를 커밋 단위로 묶어 두면 이 한계가 사라진다.

10 결론과 다음 단계

10.1 완료된 것 (증거 있음)

- SPI Slave CDC Bridge RTL(Verilog-2001, 8 모듈)의 설계와 코드 분석 — 최종 RTL 전체를 읽고 구조 문서화.
- FIFO·master·slave·reset 관련 결함 7건을 발견하고 수정했으며 각 수정의 전/후 증거를 로그 또는 코드 diff 로 제시.
- Vivado XSim 에서 최종 TB(TEST 1–10) **ALL TESTS PASSED @44,896 ns**, check 68 PASS / 0 FAIL.

10.2 완료되지 않은 것

항목	상태	비고
RTL-008 수정	미완 예정	한 줄 조건 추가로 보이나 재검증 필요
CDC-01/02 수정	미완 예정	동기기 추가
Lint	미수행	
정적 CDC / RDC	미수행	
SDC 최종 RTL 기준 재작성	미수행	현 SDC 는 다른 리비전 기준, BUFG ifdef 불일치
합성 (최종 RTL)	미수행	기존 Genus 결과는 다른 리비전
STA / LEC / P&R	미수행	
정량 Coverage	미수행	

10.3 다음 단계 (우선순위)

1. RTL-008 수정 + queue-scoreboard 방식 backlog 테스트를 최종 TB 에 포함 예정.
2. CDC-01(rx_full)과 CDC-02(busy) 동기화 수정 후 정적 CDC/RDC 도구 실행 예정.
3. 최종 RTL 로 합성 재수행: SDC 를 BUFG 인스턴스 방식에 맞추고 CDC `set_max_delay -datapath_only` 를 활성화(TUI-64 원인 확인) 예정.
4. TB 개선: 외부 포트 중심 검증, TEST 4 값 비교, TX 비대칭 값 확대, assertion/랜덤 추가, coverage 측정 예정.
5. 합성 후 STA, gate-level 시뮬레이션 또는 LEC, 이후 floorplan/PD 단계 진입 예정.

> **한 줄 결론.** 최종 RTL 은 **기능 시뮬레이션 수준에서** 설계된 10개 시나리오를 통과했고, 그 과정에서 7건의 실제 결함을 찾아 고쳤다. 그러나 RTL-008 과 두 개의 CDC 항목이 남아 있고, lint·CDC·STA·LEC·PD 는 이 증거 범위 밖이다.

A 부록 A — RTL 발췌 (최종 RTL, 설계 의도가 드러나는 부분)

A.1 sclk_gen (RTL-F L1256–1293)

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        r_toggle_phase <= 1'b0;
        o_sclk         <= 1'b0;          // 리셋 중 SCLK 정지(low)
    end else begin
        r_toggle_phase <= ~r_toggle_phase;
        if (r_toggle_phase)
            o_sclk <= ~o_sclk;           // 2 ACLK 마다 toggle -> ACLK/4
    end
end
```

A.2 reset_sync (L1214–1250)

```
always @(posedge clk or negedge arst_n) begin
    if (!arst_n) begin
        r_meta_ff  <= 1'b0;
        rst_n_sync <= 1'b0;              // assert: 비동기
    end else begin
        r_meta_ff  <= 1'b1;
        rst_n_sync <= r_meta_ff;         // release: 2 clk 동기
    end
end
```

A.3 async_fifo full / empty (L874, L912)

```
r_full  <= (w_wr_ptr_gray_next ==
            {~r_rd_ptr_gray_wclk[ADDR_WIDTH:ADDR_WIDTH-1],
              r_rd_ptr_gray_wclk[ADDR_WIDTH-2:0]});
r_empty <= (w_rd_ptr_gray_next == r_wr_ptr_gray_rclk);
```

A.4 spi_slave TX hold (RTL-006 수정)

```
assign w_tx_pop     = !i_tx_fifo_empty && !r_tx_hold_valid && !r_tx_pop_d;
assign w_tx_consume = (state == S_RECEIVE) && (bit_cnt == 3'd0) &&
                      !i_cs_n && r_tx_cur_from_hold;
if (r_tx_pop_d)   begin r_tx_hold <= i_tx_fifo_rd_data; r_tx_hold_valid <= 1'b1; end
if (w_tx_consume) r_tx_hold_valid <= 1'b0;
```

A.5 register_block RX prefetch (RTL-008 의 위치, L1032)

```
wire w_rx_prefetch = !rx_data_buf_valid && !i_rx_fifo_empty && !r_drain_active;
//                    ^ 이미 pop 요청한 다음 cycle 을 막는 !r_rx_fifo_rd_en_d 가 없음
```

A.6 spi_bridge_top — busy 동기 (CDC-02, L206–217)

```
wire w_spi_busy = w_spi_busy_master || w_spi_busy_slave;   // SCLK 도메인 조합 OR
always @(posedge ACLK) begin
    if (!aclk_rst_n) begin r_spi_busy_meta <= 1'b0; r_spi_busy_sync <= 1'b0; end
    else             begin r_spi_busy_meta <= w_spi_busy; r_spi_busy_sync <= r_spi_busy_meta; end
end
```

B 부록 B — Debugging Issue Tracker

심각도는 작성자 판단(High: 기능 손실/오동작, Med: 검증 신뢰도 또는 조건부 오동작, Low: 문서/영향 미미)입니다.

ID	모듈	심각도	내용	발견 증거	수정	재검증	상태
RTL-001	async_fifo	High	조합 루프 (full/empty ← next ptr)	코드 diff	registered flag	TEST 8/9 PASS	Closed
RTL-002	async_fifo	High	full 식 `-` → `~`	코드 diff	비트 반전	TEST 8 full@16	Closed
RTL-003	async_fifo	High	wr ptr 동기 상수 0	SS-02..05	2FF 동기	TEST 1/8/9	Closed
RTL-004	spi_master	Low	stray semicolon	코드 diff	제거	top 영향 없음	Closed
RTL-005	spi_master	Low	captured_valid 상시 1	코드 diff	S_DONE 한정	미연결 출력	Closed
RTL-006	spi_slave	High	TX 다중 pop (0x33×16)	LOG-100015	hold register	LOG-121810, TEST 2/3	Closed
RTL-007	SCLK 도메인	High	reset 중 SCLK 정지, sync reset	LOG-13 RUN9	async reset	RUN10, TEST 6/7	Closed
RTL-008	register_block	High	RX prefetch double-pop	LOG-093905	—	—	**Open**
CDC-01	top / register_block	Med	rx_full 직접 연결	RTL 코드	—	—	**Open**
CDC-02	top	Med	busy 조합 OR 후 2FF	RTL 코드	—	—	**Open**
TB-001	tb_register_block	Med	TX 쓰기 FAIL ×4 (Cadence)	SS-00	원인 미확정	—	Unknown
TB-002	tb_async_fifo	Med	공허한 PASS	SS-02	—	최종 TB TEST 5 로 개선	Mitigated
TB-003	tb top 초기	Med	MISO 재구성 불일치	LOG (5)(7)(8)	TX hold + TB 보정	RUN5 PASS	Closed (추정)
TB-004	TEST 7	Med	RX_VALID 즉시-0 검사	LOG-13 RUN11	55 ns 펄스 한정	RUN12	Closed (검사 축소)
TB-005	TEST 7	Low	주석 reset 폭 불일치	TB 분석	미수정	—	Open
LIM-01	register_block	Low	IRQ_ENABLE 출력 없음	RTL 코드	—	TEST 10 readback	Open
LIM-02	sclk_gen	Med	SCLK 생성/free-running (명세와 상이)	RTL 코드	—	—	Open (결정 필요)
LIM-03	synthesis	Med	합성 netlist 가 다른 RTL 리비전	SYN vs RTL-F	—	—	Open

C 부록 C — 최종 실행 PASS 로그 요약 (LOG-13 마지막 run)

총 68 줄 (일부 줄은 시각 접두사 없이 출력되어 **ns** 칸이 "—" 임). 라벨의 앞글자 잘림(EST7, ST10 등)은 TB 의 `check_eq` label 폭(32자) 때문이며 원문 그대로 옮겼다.

ns	로그 (PASS)	ns	로그 (PASS)
1115	`RX_DATA_REG sequential byte = 0`	36068	`EST7 CS released on reset assert = 1`
1815	`RX_DATA_REG sequential byte = 1`	36068	`TEST7 overflow reset = 0`
2735	`RX_DATA_REG sequential byte = 2`	36080	`TEST7 RX FIFO reset = 1`
3435	`RX_DATA_REG sequential byte = 3`	36080	`TEST7 TX FIFO reset = 1`
4135	`RX_DATA_REG sequential byte = 4`	37015	`TEST7 first RX byte after reset = 0`
4835	`RX_DATA_REG sequential byte = 5`	37358	`EST7 CS released on reset assert = 1`
5486	`TX_DATA_REG to MISO = 1`	37358	`TEST7 overflow reset = 0`
6246	`TX Burst byte 1 = 0x11`	37412	`TEST7 RX FIFO reset = 1`
7006	`TX Burst byte 2 = 0x22`	37412	`TEST7 TX FIFO reset = 1`
7766	`TX Burst byte 3 = 0x33`	37412	`EST7 RX valid reset (55ns pulse) = 0`
7766	`TX Burst sequence 11 -> 22 -> 33`	38355	`TEST7 first RX byte after reset = 0`
25405	`TEST4 RX FIFO FULL = 1`	39776	`recovered after repeated resets = 1`
25405	`TEST4 RX FIFO LEVEL = 10`	39870	`TEST8 initial empty = 1`
25405	`TEST4 OVERFLOW sticky = 1`	39870	`TEST8 initial not full = 0`
25405	`TEST4 OVERFLOW output = 1`	40191	`TEST8 full at 16 writes = 1`
25665	`TEST4 FULL cleared after drain = 0`	40230	`TEST8 full blocks extra write = 10`
25745	`TEST4 OVERFLOW cleared = 0`	40740	`TEST8 empty after 16 reads = 1`
—	`TEST5 FIFO Full observed`	40740	`TEST8 level zero = 0`
—	`TEST5 order[0]=a1`	41566	`TEST8 empty after wrap = 1`
—	`TEST5 order[1]=b2`	41566	`EST8 Gray single-bit transitions = 0`
—	`TEST5 order[2]=c3`	41566	`TEST8 write count = 20`
—	`TEST5 order[3]=d4`	41566	`TEST8 read count = 20`
32730	`TEST6 CS inactive in reset = 1`	41692	`TEST9 initial empty = 1`
32730	`TEST6 RX FIFO empty in reset = 1`	43988	`TEST9 writes = 80`
32730	`TEST6 TX FIFO empty in reset = 1`	43988	`TEST9 reads = 80`
32730	`TEST6 overflow cleared in reset = 0`	43988	`TEST9 FIFO empty = 1`
32730	`TEST6 RX valid cleared in reset = 0`	43988	`TEST9 FIFO level = 0`
33675	`TEST6 first RX byte after reset = 0`	44175	`ST10 unmapped address reads zero = 0`
34336	`TEST6 TX recovered after reset = 1`	44205	`TEST10 IRQ enable readback = 4`
34778	`EST7 CS released on reset assert = 1`	44235	`TEST10 IRQ disable readback = 0`
34778	`TEST7 overflow reset = 0`	44260	`TEST10 drain finishes = 0`
34784	`TEST7 RX FIFO reset = 1`	44320	`TEST10 overflow clear = 0`
34784	`TEST7 TX FIFO reset = 1`	44896	`TEST10 CS idle after frame = 1`
35725	`TEST7 first RX byte after reset = 0`	44896	`TEST10 completed SPI frame = 1`

E 부록 E — Evidence Index (핵심 파일)

파일	bytes	SHA-256 (앞 12자)	용도
`01_RTL_Versions/8bit_SPI_reset_fixed.v`	37,280	`f7cfaff92f28…`	RTL-F 최종 RTL = spi_bridge_top.v (증거 묶음 내 파일명)
`01_RTL_Versions/spi_bridge_all_in_one.v`	34,599	`64c24cb0c352…`	RTL-O 초기 RTL (RTL-001~005 의 before)
`01_RTL_Versions/spi_bridge_all_in_one_fixed.v`	35,098	`a52bcdfea3dc…`	RTL-001~005 의 after
`01_RTL_Versions/spi_tx_burst.v`	6,238	`e2525985f202…`	spi_tx_burst 시험용
`02_Testbench_Versions/tb_spi_bridge_top_test1_to_test10.v`	33,943	`96d3c95b831f…`	TB-F 최종 TB = tb_spi_bridge_top.v (증거 묶음 내 파일명)
`02_Testbench_Versions/tb_spi_bridge_top_test1_to_test7_rxvalid_check_fixed.v`	23,096	`f02587269653…`	TB-004 수정 버전
`03_Original_Logs_and_Notes/붙여넣은 텍스트(1)(20261021-131223).txt`	244,710	`d6c0751a4a82…`	LOG-13 최종 로그 (13 runs)
`03_Original_Logs_and_Notes/붙여넣은 텍스트(1)(20261021-093905).txt`	69,680	`7f143696180c…`	RTL-008 (queue TB)
`03_Original_Logs_and_Notes/붙여넣은 텍스트(1)(20261021-100015).txt`	14,617	`ac2fecad81cc…`	RTL-006 before
`03_Original_Logs_and_Notes/붙여넣은 텍스트(1)(20261021-121810).txt`	19,793	`12a40e00093e…`	RTL-006 after
`04_Screenshots/image(20261014-035520).png`	34,541	`d0269546d162…`	SS-00 tb_register_block console
`04_Screenshots/image(20261014-042041).png`	34,620	`1c2927993a2b…`	SS-02 tb_async_fifo console
`04_Screenshots/image(20261014-041717).png`	57,573	`04e2128e124c…`	그림 4-2 원본
`04_Screenshots/image(20261014-042249).png`	57,633	`c098d4363194…`	그림 4-3 원본
`04_Screenshots/image(20261021-122403).png`	250,309	`959e6a4a797a…`	TX hold 파형 (그림 4-8 ~ 4-10 계열)
`05_Specifications_and_Reviews/8-bit_SPI_Master_Slave_Async_FIFO_Specification(1).pdf`	318,601	`cd2b8b6f8dff…`	SPEC
`06_Synthesis_Reports_Optional/final.rpt`	3,083	`d56f5efcb608…`	SYN QoS 요약
`06_Synthesis_Reports_Optional/spi_bridge_top_genus_fixed.sdc`	7,542	`f480babd4ca1…`	SYN SDC

전체 목록과 전체 SHA-256 은 증거 묶음의 `README_Evidence_Index.txt` 참조. 스크린샷 그림 4-4 ~ 4-11 의 정확한 원본 파일은 파일명 시각(10/21 01:03–02:19 / 12:18–12:29)으로 좁혀지나 1:1 매핑 표는 작성하지 않았다 **미확인**.